

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN**

Nguyễn Duy Lâm - 1512090

Huỳnh Tấn Lộc - 1512007

Nguyễn Vĩnh Lương - 22120197

Huỳnh Công Minh - 1512007

Võ Hoàng Nguyên - 22120241

Nguyễn Thành Nhật - 22120241

**HỆ THỐNG HỖ TRỢ SINH VIÊN
- PHÂN HỆ QUẢN LÝ TÀI CHÍNH**

**KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CHUẨN**

GIẢNG VIÊN HƯỚNG DẪN

ThS. Phạm Hoàng Hải

ThS. Trương Phước Lộc

Tp. Hồ Chí Minh, tháng 07/2026

Lời cam đoan

Chúng tôi xin cam đoan đây là công trình nghiên cứu của chúng tôi dưới sự hướng dẫn của ThS. Phạm Hoàng Hải và ThS. Trương Phước Lộc. Các số liệu và kết quả trong báo cáo này là trung thực và không trùng lặp với các đề tài khác.

Thuyết minh chỉnh sửa đề tài

Bản thuyết minh chỉnh sửa đề tài (nếu có thay đổi tên đề tài, nội dung báo cáo, ... trong cuốn báo cáo sau bảo vệ)

Nhận xét hướng dẫn

Bản nhận xét của giảng viên hướng dẫn (có chữ ký) do giáo vụ cung cấp.

Nhận xét phản biện

Bản nhận xét của giảng viên phản biện (có chữ ký) do giáo vụ cung cấp.

Lời cảm ơn

Lời đầu tiên, chúng em xin được bày tỏ lòng biết ơn chân thành nhất đến Ban Giám hiệu cùng toàn thể quý thầy cô Trường Đại học Khoa học Tự nhiên, Đại học Quốc gia Thành phố Hồ Chí Minh, đặc biệt là các thầy cô thuộc Khoa Công nghệ Thông tin đã tận tình truyền đạt những kiến thức quý báu và tạo mọi điều kiện tốt nhất cho chúng em trong suốt quá trình học tập và rèn luyện tại trường.

Đặc biệt, chúng em xin gửi lời cảm ơn sâu sắc và chân thành nhất đến thầy **ThS. Phạm Hoàng Hải** và thầy **ThS. Trương Phước Lộc**. Trong suốt quá trình thực hiện dự án tốt nghiệp, các thầy đã luôn dành nhiều thời gian, tâm huyết tận tình hướng dẫn, định hướng khoa học và hỗ trợ chúng em vượt qua những khó khăn để hoàn thành đề tài một cách tốt nhất.

Cuối cùng, chúng em xin chân thành cảm ơn gia đình, bạn bè đã luôn ở bên cạnh động viên, hỗ trợ và đồng hành cùng chúng em trong suốt thời gian qua.

Dù đã dành nhiều thời gian và tâm huyết để hoàn thiện dự án, nhưng báo cáo chắc chắn khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những lời góp ý và chỉ dẫn quý báu từ quý thầy cô để đề tài của chúng em được hoàn thiện và phát triển tốt hơn trong tương lai.

Chúng em xin chân thành cảm ơn!

Đề cương chi tiết

Bản đề cương đã thực hiện và nộp cho giáo vụ trước đó (*phải có chữ ký của giảng viên hướng dẫn*).

Mục lục

Thuyết minh chỉnh sửa đề tài	ii
Nhận xét của GV hướng dẫn	iii
Nhận xét của GV phản biện	iv
Lời cảm ơn	v
Đề cương	vi
Mục lục	vii
Danh sách hình	xi
Danh sách bảng	xii
Danh mục từ viết tắt	xiii
Tóm tắt	xiv
1 Giới thiệu	1
1.1 Lý do chọn đề tài và bối cảnh nghiên cứu	1
1.2 Mục tiêu và phạm vi nghiên cứu	2
1.2.1 Mục tiêu nghiên cứu	2
1.2.2 Phạm vi nghiên cứu	3
1.3 Mô tả bài toán đề tài giải quyết	4

1.3.1	Bài toán quản lý tài chính cá nhân sinh viên theo phương pháp 6 lọ	4
1.3.2	Bài toán quản lý và ứng tuyển học bổng trực tuyến	4
1.3.3	Bài toán quản trị cơ sở dữ liệu học thuật của trường học	5
1.3.4	Các câu hỏi nghiên cứu	5
1.4	Những thách thức và hướng giải quyết của đề tài	6
1.4.1	Những thách thức lớn	6
1.4.2	Hướng tiếp cận giải quyết	7
1.5	Các đóng góp của đề tài	8
2	Các công trình liên quan	9
2.1	Các hệ thống quản lý tài chính cá nhân hiện có và hạn chế	9
2.2	Các giải pháp kết nối học bổng truyền thống	10
2.3	Ứng dụng trí tuệ nhân tạo trong tài chính cá nhân và gợi ý học bổng	11
2.3.1	Trong quản lý tài chính cá nhân	11
2.3.2	Trong hệ gợi ý học bổng (Scholarship Recommendation)	12
2.4	Các giải pháp thông báo đẩy thời gian thực trên nền tảng di động	13
3	Phương pháp đề xuất và Thiết kế hệ thống	14
3.1	Kiến trúc tổng thể hệ thống (System Architecture)	14
3.1.1	Kiến trúc Monorepo đa dịch vụ	14
3.1.2	Luồng truyền thông điệp và tích hợp dịch vụ	15
3.1.3	Cơ chế xác thực giao tiếp giữa các dịch vụ (Service-to-Service Auth)	16
3.2	Thiết kế cơ sở dữ liệu (Database Design)	16
3.2.1	Sơ đồ thực thể liên kết (ERD) cho mô-đun Tài chính 6 lọ	17
3.2.2	Sơ đồ thực thể liên kết (ERD) cho mô-đun Học bổng và Học thuật	18
3.3	Thiết kế chi tiết mô-đun Tài chính 6 lọ	20
3.3.1	Quy trình phân bổ nguồn thu và quản lý chi tiêu	20

3.3.2	Cơ chế lập lịch giao dịch định kỳ (Auto-transfer & Re-curring)	21
3.3.3	Cơ chế đảm bảo tính toàn vẹn số dư hũ	21
3.4	Phương pháp tích hợp Trí tuệ nhân tạo (AI Integration)	22
3.4.1	Thiết kế Trợ lý tài chính ảo thời gian thực qua stream SSE	22
3.4.2	Thuật toán phân loại giao dịch tự động bằng LLM (AI Transaction Classifier)	23
3.4.3	Thuật toán so khớp học bổng thông minh dựa trên hồ sơ sinh viên	23
3.4.4	Cấu trúc Payload giao tiếp dịch vụ (Service Communication Payload)	25
3.4.5	Cơ chế bộ nhớ đệm (Caching với TTL) tối ưu chi phí gọi LLM	27
3.5	Cơ chế thông báo di động đa kênh (Notification System)	28
3.5.1	Mô hình xử lý hàng đợi bất đồng bộ với Redis và BullMQ	28
3.5.2	Giải pháp đẩy thông báo thời gian thực qua Firebase Cloud Messaging (FCM)	28
3.5.3	Cơ chế Idempotency và khử trùng lặp cảnh báo bất thường	29
3.6	Thiết kế phân hệ Webadmin quản trị Master Data và Học bổng .	29
3.6.1	Quy trình quản lý dữ liệu học thuật và bảng điểm	30
3.6.2	Quy trình tạo, nộp và xét duyệt hồ sơ học bổng	31
4	Kết quả thí nghiệm và Hiện thực hóa	33
4.1	Môi trường triển khai và các công nghệ sử dụng	33
4.2	Hiện thực hóa giao diện ứng dụng và các luồng chức năng	34
4.2.1	Phân hệ ứng dụng di động dành cho sinh viên	34
4.2.2	Phân hệ Webadmin dành cho nhà tài trợ và ban quản trị .	35
4.2.3	Cơ chế đồng bộ trạng thái giao diện tức thời (Query Invalidation)	36
4.3	Kiểm thử và Đánh giá Hệ thống AI theo từng Giai đoạn	37

4.3.1	Giai đoạn 1: Thử nghiệm ban đầu và phát triển thô (Tháng 3 – Tháng 4/2026)	37
4.3.2	Giai đoạn 2: Tích hợp hệ thống di động và xử lý ngữ cảnh tiếng Việt (Tháng 5/2026)	39
4.3.3	Giai đoạn 3: Tối ưu hóa, kiểm soát bảo mật và nghiệm thu (Tháng 6/2026)	41
4.3.4	Giai đoạn 4: Đánh giá tích hợp API sau cải tiến	43
4.4	Đánh giá độ chính xác phân loại giao dịch tự động	44
4.5	Đánh giá hiệu quả gợi ý của thuật toán so khớp học bổng	45
4.6	Đo lường hiệu năng và độ trễ của hệ thống thông báo đẩy (FCM)	46
5	Kết luận và Hướng phát triển	47
5.1	Những kết quả đạt được của đề tài	47
5.1.1	Đánh giá so sánh hiệu quả với các giải pháp hiện hành	48
5.2	Hạn chế của hệ thống	48
5.3	Hướng phát triển tiếp theo	50
	Danh mục công trình của tác giả	52
	Tài liệu tham khảo	53
A	Ngữ pháp tiếng Việt	54
B	Ngữ pháp tiếng Nôm	55

Danh sách hình

3.1	Sơ đồ kiến trúc Monorepo đa dịch vụ của hệ thống Student360 .	15
3.2	Sơ đồ thực thể liên kết (ERD) cho mô-đun Tài chính 6 lọ	18
3.3	Sơ đồ thực thể liên kết (ERD) cho mô-đun Học bổng và Học thuật	20
3.4	Luồng xử lý và truyền stream câu trả lời của AI Agent qua SSE .	23
3.5	Mô hình xử lý hàng đợi bất đồng bộ với Redis và BullMQ	28
3.6	Sơ đồ luồng thao tác bảng điểm của sinh viên	31
4.1	Giao diện tổng quan ví 6 lọ trên thiết bị di động	35
4.2	Giao diện trò chuyện thời gian thực với trợ lý AI	35
4.3	Giao diện cấu hình phần trăm phân bổ hũ tài chính	35
4.4	Giao diện chi tiết học bổng và nộp hồ sơ trên thiết bị di động . .	35
4.5	Giao diện bảng điểm và tiến độ tín chỉ của sinh viên	35
4.6	Giao diện nhập và import điểm sinh viên	35
4.7	Giao diện quản lý danh mục trường đại học	36
4.8	Giao diện quản lý môn học và khoa/chương trình đào tạo	36
4.9	Giao diện tạo học bổng dành cho nhà tài trợ	36
4.10	Giao diện xét duyệt hồ sơ ứng tuyển học bổng	36
4.11	Đồ thị biểu diễn thời gian trễ trung bình và phân vị 95% của thông báo đẩy FCM	46

Danh sách bảng

4.1	Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 1	38
4.2	Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 2	40
4.3	Kết quả kiểm thử E2E Chat API (Non-Stream) sau cải tiến tối ưu	42
4.4	Kết quả kiểm thử tích hợp API sau cải tiến	43
4.5	Đánh giá độ chính xác phân loại giao dịch của AI theo hũ tài chính	44
4.6	Kết quả đánh giá thuật toán so khớp học bổng	45
4.7	Thời gian trễ trung bình của hệ thống gửi thông báo FCM	46
5.1	Bảng so sánh tính năng giữa Student360 và các giải pháp hiện có	49

Danh mục từ viết tắt

STT	Từ viết tắt	Giải thích
1	AI	Trí tuệ nhân tạo (Artificial Intelligence)
2	API	Giao diện lập trình ứng dụng (Application Programming Interface)
3	CRUD	Tạo - Đọc - Cập nhật - Xóa (Create, Read, Update, Delete)
4	DB	Cơ sở dữ liệu (Database)
5	ERD	Sơ đồ thực thể liên kết (Entity Relationship Diagram)
6	FCM	Dịch vụ thông báo đám mây của Firebase (Firebase Cloud Messaging)
7	GPA	Điểm trung bình tích lũy (Grade Point Average)
8	HTTP	Giao thức truyền siêu văn bản (HyperText Transfer Protocol)
9	JSON	Ký hiệu đối tượng JavaScript (JavaScript Object Notation)
10	JWT	Mã thông báo JSON Web (JSON Web Token)
11	LLM	Mô hình ngôn ngữ lớn (Large Language Model)
12	NLP	Xử lý ngôn ngữ tự nhiên (Natural Language Processing)

STT	Từ viết tắt	Giải thích
13	RBAC	Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control)
14	REST	Chuyển trạng thái đại diện (Representational State Transfer)
15	SSE	Sự kiện gửi từ máy chủ (Server-Sent Events)
16	TTL	Thời gian sống của bộ nhớ đệm (Time-To-Live)
17	UI	Giao diện người dùng (User Interface)
18	UUID	Định danh duy nhất toàn cầu (Universally Unique Identifier)
19	UX	Trải nghiệm người dùng (User Experience)

Tóm tắt

Trong đời sống giảng đường, sinh viên thường gặp khó khăn trong việc quản lý tài chính cá nhân một cách khoa học cũng như tiếp cận các thông tin học bổng phù hợp để hỗ trợ quá trình học tập. Nhằm giải quyết bài toán này, đề tài tập trung xây dựng và phát triển hệ thống hỗ trợ sinh viên toàn diện tích hợp Trí tuệ nhân tạo (AI) giúp quản lý tài chính cá nhân và tra cứu học bổng thông minh.

Hệ thống được thiết kế theo kiến trúc đa nền tảng bao gồm ứng dụng di động dành cho sinh viên và trang quản trị Web Admin dành cho người quản lý. Phân hệ tài chính cá nhân áp dụng phương pháp quản lý 6 chiếc lọ, kết hợp cùng trợ lý ảo AI để tư vấn chi tiêu và hỗ trợ lập kế hoạch tài chính hiệu quả. Bên cạnh đó, phân hệ học bổng tích hợp trợ lý AI giúp người dùng tìm kiếm, phân tích và gợi ý học bổng cá nhân hóa dựa trên hồ sơ sinh viên. Trang quản trị Web Admin được xây dựng đồng bộ nhằm phục vụ công tác quản lý thông tin học bổng, quản lý cơ cấu tổ chức trường và các khoa.

Kết quả thử nghiệm cho thấy hệ thống vận hành ổn định, giao diện trực quan, các mô hình AI phản hồi chính xác các truy vấn của người dùng với thời gian trễ thấp. Đề tài đóng góp một giải pháp công nghệ thiết thực, hỗ trợ sinh viên rèn luyện tư duy quản lý tài chính bền vững và gia tăng cơ hội tiếp cận học bổng, đồng thời cung cấp công cụ quản trị dữ liệu học tập hiệu quả cho các đơn vị quản lý.

Chương 1

Giới thiệu

1.1 Lý do chọn đề tài và bối cảnh nghiên cứu

Trong kỷ nguyên số hóa hiện nay, đời sống sinh viên tại các trường Đại học đang đối mặt với nhiều biến động và thách thức to lớn. Sự chuyển dịch kinh tế kéo theo chi phí sinh hoạt, học phí và các nhu cầu dịch vụ học tập ngày càng gia tăng. Đối với phần lớn sinh viên, việc tự chủ cuộc sống khi bước vào môi trường đại học là một bước ngoặt lớn, đi kèm với áp lực tự quản lý bản thân trên nhiều phương diện. Trong số đó, hai vấn đề nổi cộm nhất ảnh hưởng trực tiếp đến chất lượng học tập và sự phát triển bền vững của sinh viên là quản lý tài chính cá nhân và cơ hội tiếp cận các nguồn lực hỗ trợ như học bổng học thuật.

Về khía cạnh quản lý tài chính cá nhân, sinh viên thường thiếu kiến thức nền tảng và kỹ năng quản lý dòng tiền. Thói quen chi tiêu tự phát, thiếu kế hoạch, và không kiểm soát được các khoản phát sinh dẫn đến tình trạng mất cân đối tài chính thường xuyên. Điều này không chỉ gây căng thẳng tâm lý mà còn gián tiếp làm suy giảm kết quả học tập khi sinh viên phải dành nhiều thời gian để đi làm thêm trang trải cuộc sống. Phương pháp quản lý tài chính "6 cái lọ" (6 Jars) của T. Harv Eker là một mô hình nổi tiếng thế giới, giúp phân bổ thu nhập một cách khoa học vào các quỹ chi tiêu thiết yếu, đầu tư, giáo dục, tiết kiệm dài hạn, hưởng thụ và từ thiện **eker2005-6jars**. Tuy nhiên, việc áp dụng mô hình này một cách thủ công thường gặp nhiều khó khăn do tính phức tạp trong việc ghi chép, phân loại giao dịch hàng ngày và thiếu tính kỷ luật tự giác.

Về khía cạnh tiếp cận nguồn lực hỗ trợ, học bổng là một trong những mục tiêu quan trọng giúp sinh viên giảm bớt gánh nặng tài chính và tạo động lực phấn đấu trong học tập. Hiện nay, các nguồn thông tin về học bổng rất đa dạng nhưng phân tán từ nhiều nhà tài trợ, doanh nghiệp, tổ chức phi chính phủ đến chính sách nội bộ của từng trường. Sinh viên thường bỏ lỡ các cơ hội học bổng phù hợp do không tiếp cận được thông tin kịp thời hoặc hồ sơ năng lực cá nhân chưa được so khớp tối ưu với các tiêu chí xét tuyển phức tạp. Đồng thời, các nhà tài trợ cũng gặp khó khăn trong việc tìm kiếm đúng đối tượng sinh viên đáp ứng đầy đủ yêu cầu học thuật và hoàn cảnh cụ thể.

Xuất phát từ thực trạng trên, đề tài "**Hệ thống hỗ trợ sinh viên - phân hệ quản lý tài chính**" được nghiên cứu và phát triển. Đề tài hướng tới việc xây dựng một hệ sinh thái tích hợp đa nền tảng, áp dụng các giải pháp công nghệ hiện đại và trí tuệ nhân tạo (AI) để giải quyết triệt để hai bài toán cốt lõi: tự động hóa quản lý tài chính cá nhân theo mô hình 6 lọ và tối ưu hóa quy trình so khớp học bổng thông minh cho sinh viên.

1.2 Mục tiêu và phạm vi nghiên cứu

1.2.1 Mục tiêu nghiên cứu

Đề tài tập trung vào việc nghiên cứu, thiết kế và phát triển các mục tiêu cụ thể sau:

- Nghiên cứu cơ sở lý thuyết và hiện thực hóa mô hình quản lý tài chính cá nhân "6 chiếc lọ" trên ứng dụng di động, giúp sinh viên thiết lập và duy trì thói quen chi tiêu khoa học.
- Ứng dụng trí tuệ nhân tạo (AI Agent và Large Language Models - LLM) để tự động hóa quy trình phân loại giao dịch tài chính từ ngôn ngữ tự nhiên, cũng như tư vấn tài chính cá nhân thời gian thực thông qua Chatbot trợ lý ảo.

- Xây dựng phân hệ học bổng trực tuyến cho phép nhà tài trợ tạo chương trình học bổng, cấu hình điều kiện và tài liệu yêu cầu; đồng thời hỗ trợ sinh viên lưu nhập, nộp, theo dõi và hủy hồ sơ ứng tuyển theo quy trình xử lý rõ ràng.
- Xây dựng hệ thống đẩy thông báo di động (FCM Push Notification) thông minh, chạy ngầm định kỳ nhằm đưa ra các cảnh báo tài chính chủ động và thông tin học bổng khẩn cấp tới sinh viên.
- Xây dựng phân hệ quản trị dữ liệu học thuật cho phép quản trị viên quản lý danh mục trường đại học, khoa/chương trình đào tạo và môn học; đồng thời hỗ trợ sinh viên lập bảng điểm cá nhân, theo dõi GPA, tổng tín chỉ và mục tiêu học tập.

1.2.2 Phạm vi nghiên cứu

- **Về đối tượng nghiên cứu:** Tập trung vào hành vi tài chính, chi tiêu sinh hoạt thường nhật và hồ sơ năng lực học thuật của sinh viên đại học.
- **Về mặt công nghệ:** Phát triển hệ thống đa dịch vụ (Monorepo) tích hợp: Backend (NestJS, TypeScript), AI Service (FastAPI, Python, LangChain, Google Vertex API), Mobile App (Expo React Native), và Web Portal Admin (React JS, Material UI).
- **Về mặt dữ liệu:** Khai thác dữ liệu học thuật mẫu mô phỏng theo cấu trúc đào tạo tín chỉ tại các trường đại học lớn tại Việt Nam (điển hình là mô hình Đại học Quốc gia TP.HCM).

1.3 Mô tả bài toán đề tài giải quyết

1.3.1 Bài toán quản lý tài chính cá nhân sinh viên theo phương pháp 6 lọ

Bài toán yêu cầu xây dựng một mô hình số hóa 6 quỹ tài chính độc lập cho từng người dùng, đảm bảo các nguyên tắc:

- **Tính phân bổ tự động:** Khi người dùng phát sinh nguồn thu nhập mới (lương, học bổng, trợ cấp), hệ thống phải tự động phân chia dòng tiền vào 6 lọ theo tỷ lệ phần trăm được cấu hình trước.
- **Tính toàn vẹn dữ liệu tài chính:** Số dư của từng hũ tài chính phải là kết quả tính toán động từ dòng tiền thu chi thực tế trong lịch sử giao dịch để tránh sai lệch số liệu. Khi có hành vi xóa hoặc rebalance (điều chỉnh tỷ lệ hũ), hệ thống phải thực hiện các giao dịch bù trừ nội bộ để đảm bảo tổng tài sản của người dùng không bị thay đổi bất thường.
- **Hạn mức ngân sách (Budgeting):** Cho phép người dùng giới hạn chi tiêu trong từng lọ theo chu kỳ tháng để cảnh báo khi chi tiêu vượt ngưỡng thiết yếu.

1.3.2 Bài toán quản lý và ứng tuyển học bổng trực tuyến

Bài toán đặt ra là làm thế nào để số hóa toàn bộ quy trình công bố, tiếp nhận và xét duyệt học bổng giữa nhà tài trợ và sinh viên. Mỗi chương trình học bổng có thông tin riêng về giá trị tài trợ, số lượng suất, thời hạn nộp, điều kiện GPA, ngành học, trường mục tiêu và bộ tài liệu yêu cầu. Hệ thống cần đảm bảo sinh viên có thể tiếp cận thông tin rõ ràng, chuẩn bị hồ sơ trực tuyến và theo dõi trạng thái xử lý sau khi nộp.

Đối với nhà tài trợ, hệ thống cần cung cấp công quản trị để tạo và cập nhật học bổng, cấu hình điều kiện xét tuyển, định nghĩa các tài liệu bắt buộc và xem danh sách hồ sơ ứng tuyển theo từng chương trình. Đối với sinh viên, hệ thống

cần hỗ trợ lưu nháp, nộp hồ sơ, upload tài liệu minh chứng, cập nhật hồ sơ khi được yêu cầu bổ sung và hủy hồ sơ khi còn trong trạng thái cho phép.

1.3.3 Bài toán quản trị cơ sở dữ liệu học thuật của trường học

Để dữ liệu học thuật được sử dụng thống nhất giữa Webadmin và ứng dụng di động, hệ thống cần số hóa quy trình quản lý danh mục trường đại học, khoa/chương trình đào tạo và môn học. Mỗi trường có thể có nhiều chương trình đào tạo và nhiều môn học. Mỗi môn học cần lưu mã môn, tên môn, mã học phần, số tín chỉ, thang điểm và trạng thái hoạt động để phục vụ quá trình nhập điểm của sinh viên.

Đối với quản trị viên, hệ thống cần cung cấp giao diện tạo, cập nhật, xóa, tìm kiếm và import hàng loạt dữ liệu trường, khoa/chương trình đào tạo và môn học. Đối với sinh viên, hệ thống cần cho phép chọn trường và chương trình đào tạo từ danh mục có sẵn, sau đó sử dụng danh mục môn học của trường để nhập điểm, import bảng điểm, theo dõi GPA và tiến độ tín chỉ.

1.3.4 Các câu hỏi nghiên cứu

Để định hướng triển khai và đánh giá hiệu quả các giải pháp kỹ thuật đề xuất, đề tài tập trung trả lời hai câu hỏi nghiên cứu cốt lõi sau:

1. **CH1 – Phân loại giao dịch:** Làm thế nào để tự động hóa việc phân loại các giao dịch tài chính từ ngôn ngữ tự nhiên phi chuẩn (viết tắt, sai chính tả tiếng Việt) của sinh viên vào đúng hồ tài chính trong mô hình 6 lọ với độ chính xác cao mà không làm chậm trải nghiệm người dùng?
2. **CH2 – Quản lý học bổng:** Làm thế nào để thiết kế quy trình học bổng trực tuyến đảm bảo nhà tài trợ có thể công bố chương trình, sinh viên nộp hồ sơ hợp lệ, và hệ thống theo dõi được toàn bộ trạng thái xét duyệt từ lưu nháp, đã nộp, đang xét duyệt đến phê duyệt hoặc từ chối?
3. **CH3 – Hệ thống thông báo hiệu năng cao:** Kiến trúc hàng đợi tin nhắn

cần được thiết kế như thế nào để đảm bảo tính sẵn sàng cao và không gây nghẽn hệ thống khi gửi đồng loạt hàng nghìn thông báo đẩy di động (FCM) tới các sinh viên cùng lúc?

1.4 Những thách thức và hướng giải quyết của đề tài

1.4.1 Những thách thức lớn

Trong quá trình xây dựng hệ thống, nhóm nghiên cứu đối mặt với các thách thức chính sau:

1. **Độ trễ và chi phí tích hợp AI:** Việc gọi trực tiếp các API của mô hình ngôn ngữ lớn (như Gemini) có độ trễ phản hồi tương đối cao và tốn kém tài nguyên.
2. **Tính chính xác trong phân loại giao dịch:** Ngôn ngữ tự nhiên mà người dùng nhập vào khi ghi chép chi tiêu thường mang tính viết tắt, không chuẩn hóa ngữ pháp (ví dụ: "ăn trưa 50k", "cf 30k").
3. **Đồng bộ trạng thái giao diện thời gian thực:** Giao diện điện thoại di động yêu cầu cập nhật tức thì số dư của các hũ và biểu đồ báo cáo ngay sau khi người dùng thực hiện giao dịch mà không làm chậm trải nghiệm ứng dụng.
4. **Bảo mật dữ liệu và phân quyền hệ thống:** Dữ liệu tài chính cá nhân và hồ sơ học thuật/học bổng của sinh viên là các thông tin nhạy cảm. Hệ thống cần đảm bảo an toàn truy cập dữ liệu, chỉ cho phép các đối tượng thích hợp tiếp cận các tập tài nguyên tương ứng (Sinh viên truy cập app cá nhân, Nhà tài trợ quản lý hồ sơ ứng tuyển, Admin quản trị hệ thống).
5. **Đảm bảo tính nhất quán và toàn vẹn của dữ liệu giao dịch tài chính:** Khi thực hiện các tác vụ phức tạp như phân bổ thu nhập (income distri-

bution) vào 6 hũ hoặc cập nhật thông tin hũ, việc phát sinh lỗi giữa chừng hoặc race conditions có thể dẫn đến sai lệch số dư.

6. **Đồng bộ dữ liệu học thuật và bảng điểm cá nhân:** Danh mục trường, khoa/chương trình đào tạo và môn học cần nhất quán giữa Webadmin và Mobile App. Đồng thời, điểm sinh viên nhập phải được kiểm tra theo thang điểm của từng môn, tránh sai lệch GPA và tổng tín chỉ.

1.4.2 Hướng tiếp cận giải quyết

Để vượt qua các thách thức trên, đề tài đề xuất các hướng giải quyết kỹ thuật cụ thể:

- **Sử dụng kiến trúc AI Agent và Caching:** FastAPI AI Service được thiết kế độc lập, đóng vai trò điều phối (orchestration) sử dụng các Tool-calling Agent của LangChain để tối ưu hóa việc gọi mô hình. Áp dụng cơ chế lưu đệm Redis Caching với TTL (Time-to-Live) từ 5-15 phút tại Backend để lưu trữ các nhận định tài chính (AI Insights), giảm tải số lượng request trùng lặp gửi đến Gemini API.
- **Huấn luyện LLM Classifier với Rule Engine:** Sử dụng các bộ quy tắc gán cứng (Rule-based) kết hợp với kỹ thuật Few-shot Prompting để hướng dẫn mô hình Gemini phân loại các thuật ngữ viết tắt của người dùng một cách chính xác vào 6 lọ tài chính.
- **Đồng bộ giao diện qua React Query:** Áp dụng thư viện quản lý state React Query (TanStack Query) trên Expo Frontend để quản lý cơ chế Invalidation. Khi có bất kỳ thay đổi nào từ phía dữ liệu, hệ thống tự động làm mới bộ đệm cục bộ và render lại màn hình mà không cần reload toàn bộ app.
- **Xác thực JWT và cơ chế phân quyền RolesGuard:** Sử dụng JSON Web Token (JWT) để xác thực người dùng trên các API. Triển khai bộ phân quyền 'RolesGuard' chặt chẽ trong NestJS để phân tách quyền truy cập

dựa trên vai trò (Student, Donor, Admin), bảo vệ an toàn cho các endpoint nhạy cảm.

- **Quản lý dữ liệu qua Database Transactions và cơ chế recalculate số dư:** Áp dụng cơ chế transaction của TypeORM trong NestJS để đảm bảo tính nguyên tử (Atomicity) cho các tác vụ cập nhật tài chính và hồ sơ học bổng. Đồng thời, xây dựng hàm tính toán lại số dư ('recalculateJar-Balance') dựa trên lịch sử giao dịch thực tế để chống sai lệch.
- **Thiết kế cơ sở dữ liệu quan hệ chuẩn hóa cao trên PostgreSQL:** Xây dựng cấu trúc cơ sở dữ liệu với các ràng buộc khóa ngoại (Foreign Keys) chặt chẽ tại Webadmin để đồng bộ dữ liệu học thuật phân cấp từ trường học tới sinh viên, đảm bảo tính nhất quán toàn vẹn dữ liệu.

1.5 Các đóng góp của đề tài

Đề tài mang lại các đóng góp thiết thực cả về mặt học thuật lẫn ứng dụng thực tiễn:

- **Về mặt công nghệ:** Hiện thực hóa thành công một kiến trúc Monorepo đa dịch vụ ổn định, giải quyết bài toán giao tiếp bất đồng bộ và đồng bộ trạng thái giao diện thời gian thực giữa thiết bị di động, máy chủ API và lõi AI.
- **Về mặt ứng dụng tài chính:** Số hóa thành công mô hình quản lý tài chính 6 lọ, bổ sung thêm các tính năng thông minh như cảnh báo ngân sách tự động, gợi ý tái cấu trúc hũ hằng tháng (Jars Rebalancing) và trợ lý ảo tư vấn tài chính thân thiện với sinh viên.
- **Về hỗ trợ học tập:** Xây dựng phân hệ quản lý dữ liệu học thuật và bảng điểm cá nhân, giúp sinh viên theo dõi GPA, tín chỉ tích lũy, mục tiêu tín chỉ và quản lý điểm theo danh mục môn học chuẩn hóa từ nhà trường.

Chương 2

Các công trình liên quan

2.1 Các hệ thống quản lý tài chính cá nhân hiện có và hạn chế

Trong nghiên cứu về các giải pháp quản lý tài chính cá nhân hiện nay, chúng tôi phân tích các ứng dụng phổ biến trên thị trường toàn cầu và Việt Nam, tiêu biểu như Money Lover, Spendee, và Wallet by BudgetBakers. Các phần mềm này đã giải quyết tốt một số nhu cầu cơ bản của người dùng, cụ thể:

- **Money Lover:** Đây là ứng dụng quản lý tài chính nổi tiếng tại Việt Nam, hỗ trợ đắc lực việc ghi chép các khoản chi tiêu thường ngày, lập ngân sách định kỳ và quét hóa đơn thông qua ảnh chụp. Ứng dụng cung cấp các biểu đồ thống kê trực quan và hỗ trợ liên kết tài khoản ngân hàng để tự động cập nhật số dư.
- **Spendee:** Ứng dụng sở hữu thiết kế giao diện hiện đại, tối giản và tập trung mạnh vào các ví dùng chung (Shared Wallets), hỗ trợ gia đình hoặc các nhóm bạn bè cùng quản lý ngân sách chi tiêu.
- **Wallet by BudgetBakers:** Cung cấp giải pháp báo cáo chuyên sâu, hỗ trợ nhiều loại tiền tệ và tích hợp tính năng đồng bộ hóa tự động với hơn 4000 ngân hàng trên thế giới.

Tuy nhiên, các hệ thống này vẫn tồn tại những hạn chế cốt lõi sau đối với đối tượng sinh viên:

1. **Thiếu tích hợp sâu mô hình 6 lọ:** Mặc dù người dùng có thể tự tạo các danh mục chi tiêu, các ứng dụng trên chưa số hóa quy chuẩn mô hình "6 cái lọ" của T. Harv Eker **eker2005-6jars**. Người dùng phải tự thực hiện việc tính toán phần trăm phân bổ và chia nhỏ dòng tiền một cách thủ công mỗi khi có nguồn thu nhập mới.
2. **Mức độ tự động hóa bằng AI còn thấp:** Đa số các ứng dụng chỉ sử dụng các thuật toán so khớp chuỗi (string matching) hoặc biểu thức chính quy (regular expressions) đơn giản để phân loại giao dịch. Khi người dùng nhập mô tả bằng ngôn ngữ tự nhiên viết tắt hoặc sai chính tả, hệ thống thường không nhận diện được và bắt buộc người dùng chọn danh mục thủ công, gây nản lòng và bỏ dở thói quen ghi chép.
3. **Thiếu tính chủ động và tính cá nhân hóa (Proactivity):** Các ứng dụng hiện nay hoạt động theo mô hình thụ động (Passive Model), nghĩa là chỉ ghi nhận và thống kê số liệu sau khi giao dịch đã xảy ra. Hệ thống thiếu các cơ chế phân tích hành vi ngầm để đưa ra các gợi ý tái cấu trúc dòng tiền (Rebalancing) hoặc các cảnh báo chi tiêu vượt hạn mức một cách thông minh trước khi sự cố tài chính xảy ra.

2.2 Các giải pháp kết nối học bổng truyền thống

Đối với sinh viên Việt Nam, các nguồn thông tin học bổng thường được công bố qua các kênh truyền thống như:

- Trang web chính thức của Phòng Công tác Sinh viên (CTSV) hoặc Giáo vụ khoa của các trường Đại học.
- Các diễn đàn trực tuyến của sinh viên, các trang mạng xã hội (Facebook, LinkedIn).

- Các trang tin học bổng chuyên biệt (ví dụ: Scholarship Planet, YBOX).

Các phương thức truyền thông này bộc lộ nhiều hạn chế rõ rệt:

1. **Thông tin bị phân mảnh và chậm trễ:** Sinh viên phải truy cập thủ công nhiều trang web khác nhau để tìm kiếm cơ hội. Nhiều học bổng có thời gian ứng tuyển ngắn dẫn đến việc sinh viên chỉ phát hiện ra khi đã sát hạn chót nộp hồ sơ.
2. **Cơ chế lọc thông tin thô sơ:** Các trang tin học bổng hiện tại chỉ hỗ trợ lọc theo các tiêu chí cứng cơ bản (như Quốc gia, Ngành học chung). Sinh viên không thể biết chính xác hồ sơ năng lực hiện tại của mình (GPA, điểm rèn luyện, chứng chỉ tiếng Anh, hoạt động ngoại khóa) đáp ứng được bao nhiêu phần trăm điều kiện của học bổng đó.
3. **Gánh nặng cho nhà tài trợ và ban quản trị:** Quy trình xét duyệt hồ sơ ứng tuyển của sinh viên đa phần vẫn thực hiện thủ công qua email hoặc Google Forms. Việc này gây khó khăn trong việc quản lý tập trung hồ sơ, dễ xảy ra sai sót khi nhập liệu và khó phát hiện các hành vi gian lận thông tin giữa các ứng viên.

2.3 Ứng dụng trí tuệ nhân tạo trong tài chính cá nhân và gợi ý học bổng

Sự bùng nổ của các Mô hình ngôn ngữ lớn (Large Language Models - LLM) và các hệ thống đại lý thông minh (AI Agents) đã mở ra hướng giải quyết mới cho các bài toán quản lý và gợi ý thông tin.

2.3.1 Trong quản lý tài chính cá nhân

Việc ứng dụng AI giúp chuyển dịch mô hình quản lý tài chính từ thụ động sang chủ động:

- **Xử lý ngôn ngữ tự nhiên (NLP `ddien-nlp-2006`):** Sử dụng LLM để hiểu ý định của người dùng qua tin nhắn thoại hoặc văn bản mô tả ngắn gọn. AI có khả năng phân tích ngữ cảnh để trích xuất các thực thể tài chính như số tiền, mục đích chi tiêu và tự động quy về hũ tài chính tương ứng.
- **Hệ chuyên gia tư vấn:** AI Agent có thể gọi các công cụ truy vấn cơ sở dữ liệu (Tool-calling) để phân tích lịch sử chi tiêu, từ đó đưa ra các khuyến nghị tài chính cá nhân hóa như cắt giảm chi phí hưởng thụ hoặc tối ưu quỹ tiết kiệm.

2.3.2 Trong hệ gợi ý học bổng (Scholarship Recommendation)

Thay vì lọc dữ liệu theo điều kiện cứng, các nghiên cứu hiện đại áp dụng thuật toán so khớp thực thể và độ đo tương đồng văn bản:

- **Fuzzy Matching và Tokenization:** Áp dụng cho các trường dữ liệu tên trường, tên ngành hoặc kỹ năng, giúp khắc phục các lỗi gõ sai, viết tắt hoặc khác biệt về cách dịch thuật ngữ (ví dụ: "IT", "CNTT", "Information Technology"). Chỉ số độ tương đồng Jaccard **jaccard1901** và phương pháp N-gram **cavnar1994-ngram** là hai độ đo nền tảng được ứng dụng rộng rãi trong tìm kiếm thông tin không chính xác.
- **Kết hợp đa độ đo:** Trong đề tài này, thuật toán so khớp học bổng của hệ thống sử dụng tổ hợp có trọng số giữa ba độ đo chính: độ tương đồng Jaccard của tập to-ken từ vựng (trọng số 0.5), độ tương đồng Jaccard của bigram ký tự (trọng số 0.3), và tỷ lệ khớp chuỗi SequenceMatcher (trọng số 0.2). Nhờ đó, hệ thống có khả năng điều phối giữa độ nhạy cảm từ vựng cục bộ và tính rõ ràng mức ký tự, giúp giảm thiểu lỗi với các thuật ngữ viết tắt phổ biến trong môi trường học thuật Việt Nam.
- **Nhận diện ý định và phân loại gây nhầm (Intent Classification):** Sử dụng mô hình ngôn ngữ lớn Gemini **gemini-api** thông qua thư viện LangChain

langchain-docs để xây dựng AI Agent phân loại ý định học bổng của người dùng. Kết quả phân loại ý định giúp Agent điều phối đúng các Tool truy vấn, nâng cao chất lượng câu trả lời tư vấn về học bổng trong hộp thoại AI.

2.4 Các giải pháp thông báo đẩy thời gian thực trên nền tảng di động

Hệ thống thông báo đẩy (Push Notification) đóng vai trò quan trọng trong việc tăng tỷ lệ giữ chân người dùng (Retention Rate) và đảm bảo thông tin khẩn cấp được truyền tải kịp thời. Các công nghệ phổ biến bao gồm:

- **Firestore Cloud Messaging (FCM) fcm-docs:** Dịch vụ đám mây trung gian của Google, hỗ trợ gửi thông báo đẩy đến các thiết bị di động (Android và iOS) ngay cả khi ứng dụng đã bị đóng (Background/Killed state). FCM tối ưu hóa thời lượng pin của thiết bị bằng cách gom nhóm các kết nối mạng ngầm của hệ điều hành. Đây là lựa chọn chính được đề tài sử dụng để gửi thông báo học bổng và cảnh báo tài chính.
- **BullMQ và Redis Queue bullmq-docs:** Để xử lý gửi thông báo đến hàng nghìn thiết bị mà không làm nghẽn tiến trình chính của máy chủ, các hệ thống lớn áp dụng kiến trúc hàng đợi thông điệp (Message Queue). BullMQ kết hợp với Redis **redis-docs** giúp quản lý thứ tự gửi, xử lý lỗi tự động (Automatic retries) và phân bổ tài nguyên gửi tin một cách tối ưu. Đề tài áp dụng kiến trúc BullMQ cho toàn bộ hàng đợi thông báo đẩy và job quét bất thường chi tiêu của hệ thống.

Chương 3

Phương pháp đề xuất và Thiết kế hệ thống

3.1 Kiến trúc tổng thể hệ thống (System Architecture)

Hệ thống Student360 được thiết kế dựa trên mô hình monorepo đa dịch vụ nhằm đảm bảo tính phân tách trách nhiệm (Separation of Concerns), khả năng mở rộng độc lập và dễ dàng quản lý mã nguồn trong quá trình phát triển nhóm. Hệ thống bao gồm ba phân hệ chính hoạt động độc lập nhưng giao tiếp chặt chẽ với nhau thông qua mạng nội bộ và các giao thức kết nối tiêu chuẩn như HTTP REST, Server-Sent Events (SSE), và Message Queues.

3.1.1 Kiến trúc Monorepo đa dịch vụ

Cấu trúc hệ thống bao gồm bốn thành phần cốt lõi:

- **Frontend Mobile Application (Expo - React Native):** Đây là ứng dụng di động đa nền tảng dành cho đối tượng sinh viên. Nó chịu trách nhiệm hiển thị giao diện người dùng, thu thập thông tin giao dịch tài chính, cung cấp cổng tương tác trò chuyện thời gian thực với trợ lý ảo, đăng ký nhận thông báo đẩy (Push Notification) và chuẩn bị hồ sơ ứng tuyển học bổng.

Phân hệ này quản lý trạng thái dữ liệu thông qua cơ chế React Query để đảm bảo tính đồng bộ tức thời.

- **Web Admin Portal (React JS & Vite):** Phân hệ web dành cho nhà tài trợ và ban quản trị trường học. Cung cấp các công cụ quản lý cơ sở dữ liệu học thuật quy mô lớn (trường đại học, khoa/chương trình đào tạo, môn học), dashboard nhà tài trợ, chức năng tạo học bổng, cấu hình điều kiện, quản lý tài liệu yêu cầu và xét duyệt hồ sơ ứng tuyển.
- **Backend Monolith (NestJS):** Máy chủ trung tâm đóng vai trò xử lý toàn bộ logic nghiệp vụ (Business Logic), quản lý xác thực người dùng (Authentication), phân quyền dựa trên vai trò (Role-Based Access Control - RBAC), xử lý nghiệp vụ tài chính, quản lý hàng đợi BullMQ, và đóng vai trò proxy điều hướng an toàn yêu cầu tới AI Service.
- **AI Service (FastAPI):** Máy chủ trí tuệ nhân tạo chuyên biệt chịu trách nhiệm chạy các LangChain Agent kết nối với Google Gemini API, thực hiện so khớp học bổng ngữ nghĩa (Semantic Matching) và phát hiện bất thường chi tiêu (Anomaly Detection).

Hình 3.1: Sơ đồ kiến trúc Monorepo đa dịch vụ của hệ thống Student360

3.1.2 Luồng truyền thông điệp và tích hợp dịch vụ

Các phân hệ trong hệ thống giao tiếp với nhau theo hai cơ chế chính:

1. **Đồng bộ (Synchronous):** Sử dụng các RESTful API HTTP thông thường. Khi Mobile App hoặc Webadmin gửi yêu cầu cập nhật (như tạo giao dịch, sửa thông tin trường học), Backend sẽ ghi dữ liệu trực tiếp vào database PostgreSQL và trả về phản hồi lập tức.
2. **Bất đồng bộ (Asynchronous):** Đối với các tác vụ tốn nhiều tài nguyên hoặc chạy ngầm (như quét phát hiện bất thường, gửi thông báo FCM, phân

tích dữ liệu ứng tuyển), Backend sử dụng Redis làm Message Broker để đẩy các tác vụ vào hàng đợi BullMQ. Worker ngằm sẽ lấy tác vụ từ hàng đợi ra xử lý độc lập để tránh làm nghẽn tiến trình xử lý API chính của máy chủ.

3.1.3 Cơ chế xác thực giao tiếp giữa các dịch vụ (Service-to-Service Auth)

Để đảm bảo an toàn thông tin và tránh việc kẻ xấu gọi trực tiếp vào lõi AI để bào mòn chi phí API Gemini, giao tiếp giữa NestJS Backend và FastAPI AI Service được bảo vệ nghiêm ngặt bằng cơ chế xác thực nội bộ:

- API Gateway của Backend và FastAPI AI sử dụng chung một mã khóa bí mật được cấu hình qua biến môi trường `AI_SERVICE_SECRET`.
- Mỗi khi Backend gửi yêu cầu so khớp học bổng hoặc phân loại giao dịch tới FastAPI, nó sẽ đính kèm mã token này trong header của HTTP request dưới dạng Bearer Token. FastAPI AI Service sẽ giải mã và kiểm tra mã token này trước khi xử lý yêu cầu, đảm bảo chỉ có NestJS Backend được phép truy cập vào lõi AI.

3.2 Thiết kế cơ sở dữ liệu (Database Design)

Hệ thống sử dụng cơ sở dữ liệu quan hệ **PostgreSQL 15** [postgresql-docs](#) làm nền tảng lưu trữ chính cho toàn bộ dữ liệu nghiệp vụ (giao dịch tài chính, hồ sơ học bổng, dữ liệu học thuật). Bên cạnh đó, **Redis 7** [redis-docs](#) được sử dụng cho hai mục đích riêng biệt: làm Message Broker để quản lý hàng đợi BullMQ đẩy thông báo bất đồng bộ, và làm Cache lưu trữ kết quả phân tích AI Insights theo định dạng khóa `user_id:month:year` với TTL 15 phút. Việc tính toán độ tương đồng trong thuật toán so khớp học bổng được thực hiện hoàn toàn trên bộ nhớ RAM tại AI Service (Python), không phụ thuộc vào các extension như `pgvector`.

3.2.1 Sơ đồ thực thể liên kết (ERD) cho mô-đun Tài chính 6 lọ

Các bảng dữ liệu phục vụ quản lý tài chính được thiết kế tối ưu nhằm lưu vết toàn bộ lịch sử chi dùng:

- **money_jars (Các hũ tài chính):** Lưu trữ số dư và thông tin phân bổ của từng hũ.
 - id (UUID, khóa chính): Định danh duy nhất cho hũ.
 - user_id (UUID, khóa ngoại): Tham chiếu đến bảng users.
 - jar_code (VARCHAR): Phân loại hũ (nec, ffa, edu, ltss, play, give).
 - name (VARCHAR): Tên hiển thị của hũ.
 - percentage (INT): Tỷ lệ phần trăm phân bổ (Tổng 6 lọ phải bằng 100%).
 - balance (DECIMAL): Số dư hiện tại của hũ.
- **financial_transactions (Giao dịch tài chính):** Lưu vết tất cả biến động dòng tiền.
 - id (UUID, khóa chính): Định danh giao dịch.
 - user_id (UUID, khóa ngoại): Định danh người dùng thực hiện.
 - money_jar_id (UUID, khóa ngoại): Tham chiếu đến hũ nguồn hoặc hũ đích.
 - amount (DECIMAL): Số tiền thu hoặc chi.
 - type (VARCHAR): Phân loại giao dịch (income, expense, transfer).
 - category (VARCHAR): Danh mục chi tiết (ăn uống, đi lại, học tập...).

- description (TEXT): Mô tả chi tiết giao dịch.
- transaction_date (TIMESTAMPTZ): Thời điểm giao dịch phát sinh.
- budgets (**Hạn mức ngân sách**): Lưu trữ hạn mức chi tiêu tự cấu hình của người dùng.
 - id (UUID, khóa chính): Định danh ngân sách.
 - user_id (UUID, khóa ngoại): Tham chiếu người dùng.
 - money_jar_id (UUID, khóa ngoại): Lọ áp dụng hạn mức.
 - limit_amount (DECIMAL): Hạn mức chi tối đa cho phép.
 - spent_amount (DECIMAL): Số tiền thực tế đã chi trong chu kỳ.
 - month (INT), year (INT): Khoảng thời gian áp dụng hạn mức.

Hình 3.2: Sơ đồ thực thể liên kết (ERD) cho mô-đun Tài chính 6 lọ

3.2.2 Sơ đồ thực thể liên kết (ERD) cho mô-đun Học bổng và Học thuật

Các bảng dữ liệu nền tảng học thuật và học bổng bao gồm các liên kết khóa ngoại chặt chẽ, được chi tiết hóa các trường dữ liệu để phục vụ kiểm soát nghiệp vụ:

- universities (**Trường đại học**): Lưu danh mục trường dùng cho Webadmin và Mobile App. Các trường chính gồm id, name, code, type, city, status, created_at và updated_at.
- training_programs (**Khoa/Chương trình đào tạo**): Lưu danh mục khoa hoặc chương trình đào tạo thuộc từng trường. Các trường chính gồm id, university_id, program_code, program_name, description, status, created_at và updated_at.

- **university_subjects (Môn học):** Lưu danh mục môn học theo từng trường. Các trường chính gồm id, university_id, subject_code, subject_name, course_code, credits, grade_scale, description, status, created_at và updated_at.
- **student_academic_profiles (Hồ sơ học thuật sinh viên):** Lưu thông tin học thuật của sinh viên, gồm user_id, university, student_code, education_program, degree_level, faculty, major, minor, program_type, gpa, enrollment_year, expected_graduation_year, current_status, current_year và current_semester.
- **student_grades (Bảng điểm sinh viên):** Lưu điểm từng môn do sinh viên nhập hoặc import. Dữ liệu gắn với sinh viên và môn học, gồm subject_id, grade và student_note. Hệ thống sử dụng thông tin credits và grade_scale từ university_subjects để kiểm tra điểm hợp lệ và tính GPA.
- **gradebook_targets (Mục tiêu bảng điểm):** Lưu mục tiêu học tập cá nhân của sinh viên, gồm target_credits và target_note, phục vụ dashboard tiến độ tín chỉ.
- **scholarships (Học bổng):** Lưu trữ thông tin chương trình học bổng do nhà tài trợ công bố. Các trường chính gồm id, name, description, provider, provider_id, amount, currency, quantity, minimum_gpa, minimum_gpa_scale, target_majors, target_universities, application_deadline, result_announcement_date, benefits, contact_email, contact_phone, official_website và is_active.
- **scholarship_requirements (Điều kiện học bổng):** Lưu các điều kiện chi tiết do nhà tài trợ cấu hình cho từng học bổng, gồm title, description, is_required và sort_order.

- **scholarship_documents (Tài liệu yêu cầu):** Định nghĩa các loại tài liệu sinh viên cần nộp, gồm `document_name`, `document_type`, `is_required`, `max_file_size_mb` và `sample_url`.
- **student_scholarships (Hồ sơ ứng tuyển):** Lưu hồ sơ ứng tuyển của sinh viên cho từng học bổng. Bảng có ràng buộc duy nhất trên cặp `user_id` và `scholarship_id` để ngăn nộp trùng. Trạng thái hồ sơ gồm `draft`, `submitted`, `under_review`, `need_modification`, `approved`, `rejected`, `awarded` và `cancelled`.
- **student_scholarship_documents (Tài liệu hồ sơ):** Lưu các file sinh viên đã upload cho từng loại tài liệu yêu cầu, gồm `file_url`, `upload_date`, `status` và `reviewer_note`.
- **student_scholarship_history (Lịch sử trạng thái hồ sơ):** Ghi nhận các lần chuyển trạng thái của hồ sơ, gồm `from_status`, `to_status`, `note`, `changed_by` và `created_at`.

Hình 3.3: Sơ đồ thực thể liên kết (ERD) cho mô-đun Học bổng và Học thuật

3.3 Thiết kế chi tiết mô-đun Tài chính 6 lọ

3.3.1 Quy trình phân bổ nguồn thu và quản lý chi tiêu

Quy trình phân bổ dòng tiền được thực hiện tự động nhằm giảm thiểu thao tác nhập liệu thủ công cho sinh viên:

1. Khi người dùng nhận một khoản thu mới (ví dụ: Học bổng khuyến khích học tập trị giá 10.000.000 VNĐ) và chọn hình thức phân bổ tự động.
2. Hệ thống gọi API `POST /distribute-income`. Lỗi Backend NestJS sẽ truy vấn cấu hình tỷ lệ phần trăm của 6 lọ hiện tại của người dùng (ví dụ cấu hình chuẩn: 55-10-10-10-10-5).

3. Hệ thống tự động tính toán số tiền tương ứng cho từng lọ (ví dụ: hũ nhu cầu thiết yếu nhận 5.500.000 VNĐ, hũ tự do tài chính nhận 1.000.000 VNĐ...) và thực hiện ghi đồng thời (Batch write) 6 lệnh cộng tiền tương ứng vào cơ sở dữ liệu.

3.3.2 Cơ chế lập lịch giao dịch định kỳ (Auto-transfer & Recurring)

Để quản lý các khoản phí cố định phát sinh hàng tháng, hệ thống cho phép người dùng thiết lập lịch giao dịch:

- Người dùng khai báo các khoản chi tiêu định kỳ (như Tiền nhà trọ, tiền mạng internet) kèm ngày thanh toán và hạn mức.
- Một Cron Job ngầm tại Backend chạy vào lúc 00:05 mỗi ngày để quét các lịch trình kích hoạt trong ngày hiện tại, tự động tạo giao dịch expense từ lọ được chỉ định và gửi thông báo nhắc nhở tới thiết bị của người dùng.

3.3.3 Cơ chế đảm bảo tính toàn vẹn số dư hũ

Nhằm ngăn ngừa hiện tượng sai lệch hoặc thất thoát số dư do lỗi bất đồng bộ mạng hoặc lỗi phần mềm, hệ thống áp dụng nguyên tắc:

- **Số dư động (Dynamic Balance):** Số dư hiển thị trên giao diện của từng hũ không được lưu trữ tĩnh mà luôn được tính toán lại bằng công thức tích lũy từ lịch sử các dòng tiền thực tế:

$$\text{Số dư hũ} = \sum(\text{Thu vào}) - \sum(\text{Chi ra}) \pm \sum(\text{Chuyển quỹ})$$

- **Hành vi khi xóa hũ tài chính:** Khi người dùng muốn xóa một hũ tài chính tùy chỉnh, hệ thống không chỉ đơn giản là xóa dòng dữ liệu trong DB. Nếu hũ đó có số dư khác 0, hệ thống bắt buộc người dùng chọn một hũ đích để nhận lại tiền. Sau đó, hệ thống thực thi một Transaction DB

để tạo đồng thời một cặp giao dịch bù trừ: trừ tiền ở hũ nguồn (Giao dịch expense) và cộng tiền vào hũ đích (Giao dịch income) để đảm bảo tổng tài sản của người dùng hoàn toàn bất biến.

3.4 Phương pháp tích hợp Trí tuệ nhân tạo (AI Integration)

Lỗi AI được tích hợp sâu vào hệ thống qua FastAPI AI Service, khai thác năng lực suy luận của mô hình Large Language Model (Gemini 1.5 Flash/Pro) kết hợp thư viện LangChain để xây dựng hệ thống tác nhân (AI Agents).

3.4.1 Thiết kế Trợ lý tài chính ảo thời gian thực qua stream SSE

Để mang lại trải nghiệm trò chuyện mượt mà giống như các ứng dụng Chat-GPT hay Gemini thương mại, hệ thống sử dụng giao thức Server-Sent Events (SSE):

- Khi sinh viên đặt câu hỏi (ví dụ: "Tháng này tôi đã tiêu bao nhiêu tiền ăn?"), ứng dụng di động sẽ kết nối tới Backend thông qua endpoint `/ai/chat`.
- Backend hoạt động như một Proxy chuyển tiếp request này sang cổng `/api/v1/chat/stream` của FastAPI AI Core.
- AI Service sử dụng LangChain Agent để điều phối câu hỏi. Agent phân tích câu hỏi và quyết định gọi các Tool nghiệp vụ (như `get_jar_balance`, `get_transaction_history`) để truy vấn số liệu thực tế từ Backend.
- Sau khi nhận kết quả từ database, Agent truyền nội dung trả lời về cho Backend dưới dạng các token văn bản nối tiếp nhau. Backend lập tức đẩy

các token này về Mobile App thông qua kênh SSE kết nối sẵn, giúp chữ hiển thị chạy ra màn hình ngay lập tức thay vì phải đợi toàn bộ câu trả lời hoàn tất.

Hình 3.4: Luồng xử lý và truyền stream câu trả lời của AI Agent qua SSE

3.4.2 Thuật toán phân loại giao dịch tự động bằng LLM (AI Transaction Classifier)

Khi người dùng nhập mô tả giao dịch (ví dụ: "mua sách 150k" hoặc "đi cho 80k"), hệ thống gọi endpoint `/api/ai/6jars/classify`.

- Hệ thống gửi văn bản mô tả và số tiền tới LLM Classifier.
- Mô hình sử dụng Few-shot Prompting kết hợp cơ chế gán nhãn để phân tích ngữ nghĩa, tự động suy luận ra:
 - "mua sách" → Loại đề xuất: `education` (Học tập), Danh mục: Sách vở/Học cụ.
 - "đi cho" → Loại đề xuất: `necessity` (Nhu cầu thiết yếu), Danh mục: Thực phẩm/Ăn uống.
- **Cơ chế Override (Học hành vi người dùng):** Nếu AI phân loại sai hũ (ví dụ đề xuất loại `Hưởng thụ` nhưng người dùng sửa thủ công thành loại `Cho đi` trước khi lưu), hệ thống sẽ gọi API `/override` để lưu cập dữ liệu sửa lỗi này vào bảng học tập của AI. Lần sau khi gặp từ khóa tương tự, AI sẽ ưu tiên lựa chọn sửa đổi của người dùng.

3.4.3 Thuật toán so khớp học bổng thông minh dựa trên hồ sơ sinh viên

Thuật toán so khớp học bổng được triển khai trong hàm `_score_profile_match` tại AI Service, sử dụng tổ hợp có trọng số của ba độ đo tương đồng văn bản.

Thuật toán hoạt động hoàn toàn trên bộ nhớ RAM của Python AI Service, không phụ thuộc vào cơ sở dữ liệu vector.

Bước 1: Chuẩn hóa và gộp hồ sơ sinh viên. Hệ thống gộp toàn bộ các trường dữ liệu có giá trị (ngành học, trường, kỹ năng, hoạt động ngoại khóa) của sinh viên thành một chuỗi văn bản phẳng dùng hàm `_build_profile_text`, sau đó áp dụng *Unicode NFC normalization* để chuẩn hóa đồng nhất các ký tự tiếng Việt trên các hệ điều hành khác nhau.

Bước 2: Tính độ tương đồng Jaccard trên tập token từ vựng (trọng số 0.5). Chỉ số Jaccard `jaccard1901` được tính giữa tập hợp token từ vựng của hồ sơ A và mô tả học bổng B :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (3.1)$$

Trong đó A là tập các token thu được sau khi tokenize văn bản hồ sơ sinh viên, B là tập token từ tiêu đề và mô tả học bổng.

Bước 3: Tính độ tương đồng Jaccard trên Bigram ký tự (trọng số 0.3). Tập hợp n -gram ký tự độ dài $n = 2$ (Bigrams) được định nghĩa theo `cavnar1994-ngram`:

$$N(S, 2) = \{s_i s_{i+1} \mid i = 1, 2, \dots, |S| - 1\} \quad (3.2)$$

Ví dụ với chuỗi $S = \text{"cntt"} \rightarrow N(S, 2) = \{\text{"cn"}, \text{"nt"}, \text{"tt"}\}$. Độ tương đồng Bi-gram giúp hệ thống nhận diện các từ viết tắt hoặc sai chính tả.

Bước 4: Tính tỷ lệ khớp chuỗi SequenceMatcher (trọng số 0.2). Sử dụng mô-đun `difflib.SequenceMatcher` của Python để đo độ tương đồng của dãy ký tự giữa văn bản hồ sơ và tên học bổng.

Bước 5: Kết hợp có trọng số. Điểm số khớp tổng hợp cuối cùng được tính theo công thức:

$$\text{score} = 0.5 \times J_{\text{token}}(A, B) + 0.3 \times J_{\text{bigram}}(A, B) + 0.2 \times \text{Seq}(A, B) \quad (3.3)$$

Giải thuật được mô tả chính thức như sau:

Algorithm 1 Thuật toán so khớp học bổng dựa trên hồ sơ sinh viên

```

1: procedure SCOREPROFILEMATCH(Profile, Scholarship)
2:   profileText  $\leftarrow$  BUILDPROFILETEXT(Profile)
3:   profileText  $\leftarrow$  NFCNORMALIZE(profileText)
4:   scholText  $\leftarrow$  Scholarship.name + Scholarship.eligibility_criteria
5:   profileTokens  $\leftarrow$  TOKENIZE(profileText)
6:   scholTokens  $\leftarrow$  TOKENIZE(scholText)
7:   if profileTokens =  $\emptyset$  or scholTokens =  $\emptyset$  then
8:     return 0.0
9:   tokenScore  $\leftarrow$   $J(\textit{profileTokens}, \textit{scholTokens})$  ▷ Công thức (3.1)
10:  profileBigrams  $\leftarrow$   $N(\textit{profileText}, 2)$  ▷ Công thức (3.2)
11:  scholBigrams  $\leftarrow$   $N(\textit{scholText}, 2)$ 
12:  bigramScore  $\leftarrow$   $J(\textit{profileBigrams}, \textit{scholBigrams})$ 
13:  seqScore  $\leftarrow$  SEQUENCEMATCHER(profileText, Scholarship.name)
14:  score  $\leftarrow$   $0.5 \times \textit{tokenScore} + 0.3 \times \textit{bigramScore} + 0.2 \times \textit{seqScore}$ 
15:  return ROUND(score, 6)

```

3.4.4 Cấu trúc Payload giao tiếp dịch vụ (Service Communication Payload)

Để thực thi các nhiệm vụ AI tích hợp, hệ thống sử dụng các gói dữ liệu JSON chuẩn hóa được trao đổi qua giao thức HTTP REST giữa NestJS Backend và FastAPI AI Service.

1. Payload phân loại giao dịch tài chính (POST /ai/classify-transaction):

Khi người dùng nhập mô tả giao dịch bằng ngôn ngữ tự nhiên, Backend sẽ gửi payload có cấu trúc như sau:

```

1 {
2   "userId": "d3b07384-d113-49cd-a5d6-84860d5b4a2f",
3   "rawMessage": "toi mua vo ghi bai het 15k bang tien mat",
4   "currentBalance": 450000.00
5 }

```

Listing 3.1: Payload yêu cầu phân loại giao dịch

FastAPI AI Service sau khi xử lý sẽ trả về kết quả phân loại và trích xuất thực thể hành động dưới dạng:

```
1 {
2   "intent": "personal_finance",
3   "confidence": 0.942,
4   "action": {
5     "type": "expense",
6     "amount": 15000.0,
7     "jarCode": "edu",
8     "category": "hoc_tap",
9     "currency": "VND"
10  },
11  "rawResponse": "Da ghi nhan khoan chi 15,000 VND vao lo Giao duc."
12 }
```

Listing 3.2: Response kết quả phân loại giao dịch từ AI

2. Payload yêu cầu so khớp học bổng (POST /ai/match-scholarships):
Để thực hiện so khớp học bổng, Backend gửi toàn bộ hồ sơ học thuật và ngoại khóa của sinh viên qua payload cấu trúc như sau:

```
1 {
2   "studentProfile": {
3     "gpa": 3.65,
4     "faculty": "Cong nghe thong tin",
5     "skills": ["Python", "Machine Learning", "LaTeX"],
6     "activities": ["Tham gia nghien cuu khoa hoc", "CLB Tinh nguyen"]
7   },
8   "scholarshipsList": [
9     {
10      "id": "a901ff21-8255-4cc2-9854-9442ef5432ab",
11      "name": "Hoc bong Vingroup khoa hoc cong nghe",
12      "eligibility": "Sinh vien nganh CNTT hoac Dien tu vien thong co GPA
13      tu 3.5 tro len va co thanh tich nghien cuu khoa hoc."
14    }
15  ]
16 }
```

Listing 3.3: Payload thông tin hồ sơ sinh viên để so khớp học bổng

AI Service sẽ trả về danh sách các học bổng kèm điểm số matching tương ứng:

```
1 {
2   "matchedScholarships": [
```

```

3  {
4    "scholarshipId": "a901ff21-8255-4cc2-9854-9442ef5432ab",
5    "matchingScore": 0.814322,
6    "reasons": [
7      "Nganh hoc Cong nghe thong tin khop voi CNTT trong tieu chi.",
8      "Diem GPA 3.65 dat yeu cau tren 3.5.",
9      "Co ky nang nghien cuu khoa hoc."
10   ]
11 }
12 ]
13 }

```

Listing 3.4: Response gợi ý học bổng tương thích từ AI

3.4.5 Cơ chế bộ nhớ đệm (Caching với TTL) tối ưu chi phí gọi LLM

Để giải quyết thách thức về chi phí API và độ trễ phản hồi:

- Khi người dùng mở màn hình báo cáo, hệ thống cần hiển thị một khối "Góc nhìn AI" (AI Insights) nhận định tình hình tài chính của tháng.
- Thay vì gọi mô hình Gemini phân tích lại từ đầu mỗi khi người dùng chuyển đổi tab, Backend NestJS sẽ lưu trữ đoạn văn bản nhận xét của AI vào bộ nhớ đệm Redis Cache.
- Khóa cache (Cache Key) được định danh duy nhất theo định dạng `user_id:month:year`. Thời gian tồn tại của cache (TTL) được cấu hình là 15 phút.
- Bất kỳ yêu cầu tải báo cáo nào trong khoảng thời gian này sẽ được Backend trả về ngay lập tức từ Redis mà không tốn chi phí gọi LLM, rút ngắn thời gian phản hồi từ khoảng 8 giây xuống còn dưới 50 mili giây.

3.5 Cơ chế thông báo di động đa kênh (Notification System)

Hệ thống thông báo đẩy được thiết kế để hoạt động ổn định dưới tải trọng cao và đảm bảo tính thời gian thực.

3.5.1 Mô hình xử lý hàng đợi bất đồng bộ với Redis và BullMQ

Luồng xử lý thông báo được tổ chức như sau:

1. Khi một sự kiện cần gửi thông báo xảy ra (ví dụ: Học bổng mới được duyệt hoặc phát hiện chi tiêu bất thường).
2. NestJS Service đóng vai trò Producer tạo ra một Job chứa thông tin tiêu đề, nội dung và danh sách các token thiết bị di động của người nhận, sau đó đẩy Job này vào hàng đợi Redis thông qua BullMQ.
3. Một cụm các tiến trình Worker chạy ngầm sẽ liên tục lấy các Job ra khỏi hàng đợi Redis để xử lý. Việc xử lý bất đồng bộ giúp máy chủ API chính không bị block kết nối mạng khi phải thực thi các tác vụ gửi tin nhắn IO-intensive ra bên ngoài.

Hình 3.5: Mô hình xử lý hàng đợi bất đồng bộ với Redis và BullMQ

3.5.2 Giải pháp đẩy thông báo thời gian thực qua Firebase Cloud Messaging (FCM)

Các Worker sử dụng thư viện `firebase-admin` của Firebase SDK để kết nối tới máy chủ Google FCM:

- Hệ thống xây dựng cấu trúc tải tin (Payload) tối ưu cho từng nền tảng hệ điều hành.

- Đối với Android: Đặt độ ưu tiên cao (high), thời gian sống của tin nhắn (TTL) là 24 giờ, sử dụng kênh thông báo chuyên biệt `student360_notifications` để kích hoạt âm thanh mặc định và chế độ rung trên thiết bị.
- Đối với iOS (APNs): Đính kèm tiêu đề và nội dung tin nhắn trong cấu trúc APNs headers với độ ưu tiên cao (`apns-priority: 10`) để đánh thức thiết bị ở chế độ nền.

3.5.3 Cơ chế Idempotency và khử trùng lặp cảnh báo bất thường

Khi chạy công việc (job) quét phát hiện chi tiêu bất thường định kỳ vào cuối ngày, hệ thống có thể phát hiện cùng một hồ danh cho nhu cầu thiết yếu của người dùng liên tục bị vượt hạn mức hoặc tăng vọt đột biến. Để tránh làm phiền người dùng bằng việc gửi lặp đi lặp lại một thông báo cảnh báo:

- Hệ thống áp dụng cơ chế loại bỏ trùng lặp (Deduplication) dựa trên khóa Idempotency.
- Khóa này được tính toán dựa trên tổ hợp: `user_id + module_type + alert_type + target_id + month`.
- Trước khi insert một cảnh báo bất thường mới vào bảng `ai_anomaly_alerts` và kích hoạt job gửi Push Notification, Worker sẽ kiểm tra trong database xem đã tồn tại cảnh báo nào có cùng khóa Idempotency trong tháng hiện tại chưa. Nếu đã có, hệ thống sẽ bỏ qua việc tạo mới và không gửi thêm thông báo nữa.

3.6 Thiết kế phân hệ Webadmin quản trị Master Data và Học bổng

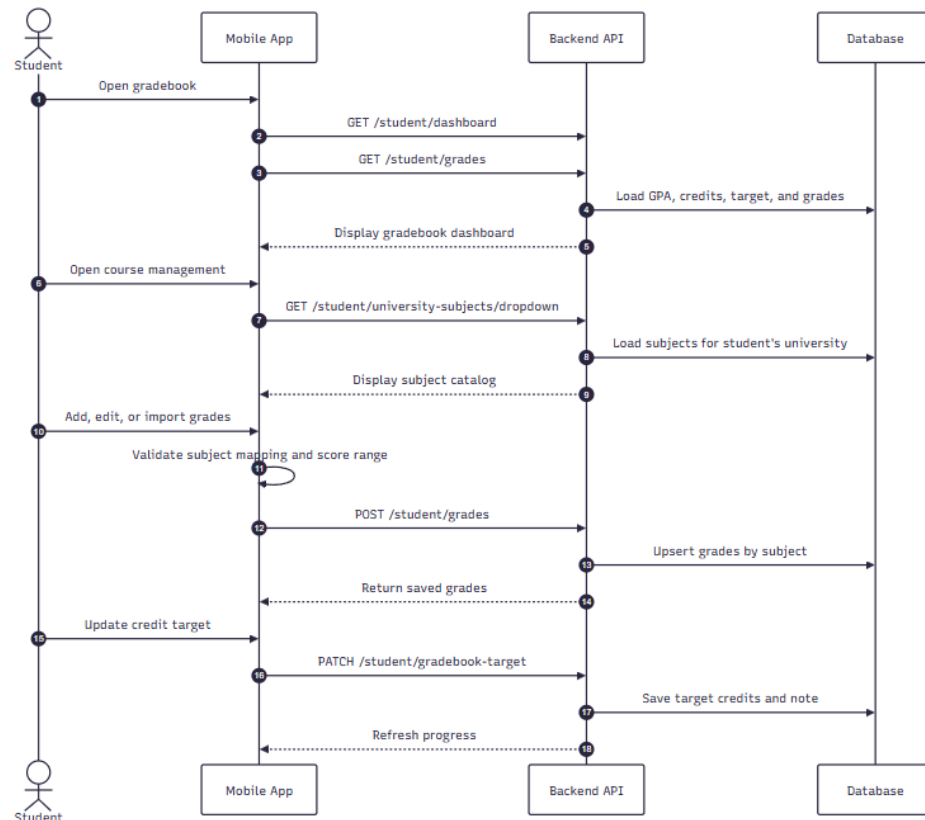
Phân hệ Webadmin được xây dựng trên nền tảng React JS, phục vụ công tác quản lý của Ban giám hiệu trường học và các Nhà tài trợ học bổng.

3.6.1 Quy trình quản lý dữ liệu học thuật và bảng điểm

Quy trình được thiết kế theo hai nhóm người dùng chính: quản trị viên và sinh viên.

1. **Quản trị viên quản lý trường:** Webadmin lấy danh sách trường qua GET /admin/universities, tạo mới qua POST /admin/universities, cập nhật qua PUT /admin/universities/{id} và xóa qua DELETE /admin/universities/{id}.
2. **Quản trị viên quản lý khoa/chương trình đào tạo:** Với từng trường, Webadmin lấy danh sách khoa/chương trình đào tạo qua GET /admin/universities/{id}/training-programs, thêm mới qua POST /admin/universities/{id}/training-programs, cập nhật qua PUT /admin/training-programs/{id}, xóa qua DELETE /admin/training-programs/{id} và import hàng loạt qua POST /admin/universities/{id}/training-programs/bulk.
3. **Quản trị viên quản lý môn học:** Với từng trường, Webadmin lấy danh sách môn qua GET /admin/universities/{id}/subjects, thêm mới qua POST /admin/universities/{id}/subjects, cập nhật qua PUT /admin/subjects/{id}, xóa qua DELETE /admin/subjects/{id} và import hàng loạt qua POST /admin/universities/{id}/subjects/bulk.
4. **Sinh viên chọn thông tin học thuật:** Mobile App lấy danh sách trường qua GET /universities/dropdown và danh sách khoa/chương trình đào tạo theo trường qua GET /universities/{universityId}/training-programs. Thông tin này được dùng trong hồ sơ đăng ký và hồ sơ cá nhân.
5. **Sinh viên xem danh mục môn học:** Mobile App lấy danh mục môn của trường qua GET /student/university-subjects/dropdown. Mỗi môn có mã môn, mã học phần, số tín chỉ và thang điểm để phục vụ nhập điểm.
6. **Sinh viên nhập hoặc import điểm:** Sinh viên thêm hoặc cập nhật điểm qua POST /student/grades. File TXT/CSV được parse ở Mobile App, map môn theo subject_id, subject_code hoặc course_code, cho phép chỉnh sửa trước khi lưu.

7. **Sinh viên quản lý bảng điểm:** Sinh viên xem danh sách điểm qua GET /student/grades, xóa điểm qua DELETE /student/grades/{gradeId}, xem dashboard qua GET /student/dashboard và cập nhật mục tiêu tín chỉ qua PATCH /student/gradebook-target.



Hình 3.6: Sơ đồ luồng thao tác bảng điểm của sinh viên

3.6.2 Quy trình tạo, nộp và xét duyệt hồ sơ học bổng

Quy trình học bổng được thiết kế theo hai vai trò chính: nhà tài trợ và sinh viên.

1. **Nhà tài trợ tạo học bổng:** Nhà tài trợ đăng nhập Webadmin, nhập thông tin chương trình học bổng và gửi yêu cầu POST /scholarships. Back-

end lưu bản ghi vào bảng scholarships, trong đó provider_id liên kết học bổng với tổ chức tài trợ trong bảng organizations.

2. **Cấu hình điều kiện và tài liệu:** Sau khi tạo học bổng, nhà tài trợ cấu hình các điều kiện xét tuyển qua POST /scholarships/{id}/requirements và danh sách tài liệu yêu cầu qua POST /scholarships/{id}/documents.
3. **Sinh viên xem và chuẩn bị hồ sơ:** Ứng dụng di động lấy danh sách học bổng qua GET /scholarships và chi tiết từng học bổng qua GET /scholarships/{id}. Sinh viên nhập CCCD, GPA, ghi chú và upload các tài liệu minh chứng theo danh sách yêu cầu.
4. **Lưu nháp hoặc nộp hồ sơ:** Sinh viên gửi hồ sơ qua POST /scholarships/register. Nếu chọn lưu nháp, hệ thống tạo hồ sơ ở trạng thái draft; nếu nộp chính thức, hồ sơ chuyển sang submitted.
5. **Kiểm tra hợp lệ ở Backend:** Backend kiểm tra hạn nộp, tình trạng hồ sơ cá nhân, GPA tối thiểu, định dạng CCCD và ràng buộc không nộp trùng. Việc tạo hoặc cập nhật hồ sơ, lưu tài liệu và ghi lịch sử trạng thái được thực hiện trong cùng một database transaction.
6. **Nhà tài trợ xét duyệt:** Webadmin lấy danh sách hồ sơ theo tổ chức qua GET /student-scholarships/organization/{organization_id}. Nhà tài trợ xem chi tiết hồ sơ, tài liệu đính kèm, GPA, thông tin học thuật và cập nhật trạng thái qua PUT /student-scholarships/{id}/status.
7. **Sinh viên theo dõi kết quả:** Sinh viên xem các hồ sơ đã nộp qua GET /scholarships/my-scholarships, xem chi tiết từng hồ sơ qua GET /scholarships/my-scholarships/{id}, và có thể hủy hồ sơ khi còn ở trạng thái cho phép qua DELETE /scholarships/my-scholarships/{id}.

Chương 4

Kết quả thí nghiệm và Hiện thực hóa

4.1 Môi trường triển khai và các công nghệ sử dụng

Hệ thống Student360 được hiện thực hóa và kiểm thử trên môi trường cục bộ và nền tảng điện toán đám mây với các thông số cấu hình cụ thể như sau:

- **Hệ điều hành thử nghiệm:** macOS Sequoia v15.5 / Ubuntu 22.04 LTS.
- **Môi trường máy chủ Backend:** Node.js v20.11.0, NestJS v10.3.0 [nestjs-docs](#), TypeORM v0.3.20 [typeorm-docs](#).
- **Môi trường máy chủ AI:** Python v3.11.8, FastAPI v0.110.0 [fastapi-docs](#), LangChain v0.1.12 [langchain-docs](#), Uvicorn v0.28.0.
- **Công nghệ cơ sở dữ liệu:** PostgreSQL v15.3 [postgresql-docs](#), Redis v7.2.4 [redis-docs](#) (làm cache và message broker cho BullMQ [bullmq-docs](#)).
- **Môi trường Mobile App:** Expo SDK v50 (React Native v0.73) [reactnative-expo](#), kết hợp Tamagui v1.90 cho hệ thống Design System.
- **Môi trường Webadmin:** React JS v18.2, Material UI (MUI) v5.15, Vite v5.1.

- **Mô hình AI:** Google Gemini 1.5 Flash / Pro (thông qua Vertex AI API) `gemini-api`.

4.2 Hiện thực hóa giao diện ứng dụng và các luồng chức năng

4.2.1 Phân hệ ứng dụng di động dành cho sinh viên

Giao diện ứng dụng di động được thiết kế hướng tới sự trực quan, trẻ trung và tối ưu hóa trải nghiệm người dùng (UX):

- **Màn hình Tổng quan ví (Wallet Overview):** Hiển thị tổng số dư khả dụng, chỉ số sức khỏe tài chính (Financial Health Score) tính toán động từ thói quen chi tiêu của sinh viên, kèm theo biểu đồ hình tròn biểu diễn tỷ lệ phân bổ của các hũ tài chính (bao gồm chi tiêu thiết yếu, tự do tài chính, học tập, tiết kiệm dài hạn, hưởng thụ và cho đi).
- **Màn hình Chi tiết hũ:** Khi sinh viên chọn một hũ cụ thể, giao diện hiển thị số dư riêng biệt của hũ đó, danh sách lịch sử giao dịch tương ứng và một biểu đồ đường (Line Chart) thể hiện xu hướng biến động số dư theo tháng.
- **Màn hình Trò chuyện AI (AI Chat Screen):** Cung cấp giao diện chat thời gian thực. Tin nhắn phản hồi của AI trợ lý sẽ hiển thị dạng stream từ trái qua phải nhờ tích hợp luồng SSE.
- **Màn hình Cấu hình phân bổ lọ (Jar Allocation):** Cho phép sinh viên kéo các thanh trượt (slider) hoặc nhập phần trăm thủ công để điều chỉnh tỷ lệ 6 lọ tài chính.
- **Màn hình Học bổng và Nộp hồ sơ:** Hiển thị danh sách học bổng đang mở, chi tiết điều kiện ứng tuyển, giá trị học bổng, hạn nộp, tài liệu bắt buộc và thông tin liên hệ. Sinh viên có thể mở form ứng tuyển, nhập

CCCD, GPA, ghi chú, upload tài liệu minh chứng, lưu nháp hoặc nộp hồ sơ chính thức.

- **Màn hình Bảng điểm sinh viên:** Hiển thị GPA hiện tại, tổng tín chỉ đã học, mục tiêu tín chỉ và danh sách môn đã nhập điểm. Sinh viên có thể thêm môn từ danh mục của trường, nhập điểm theo thang điểm của môn, chỉnh sửa hoặc xóa điểm, import bảng điểm từ file TXT/CSV và kiểm tra dữ liệu trước khi lưu.

Hình 4.1: Giao diện tổng quan ví 6 lọ trên thiết bị di động

Hình 4.2: Giao diện trò chuyện thời gian thực với trợ lý AI

Hình 4.3: Giao diện cấu hình phần trăm phân bổ hũ tài chính

Hình 4.4: Giao diện chi tiết học bổng và nộp hồ sơ trên thiết bị di động

Hình 4.5: Giao diện bảng điểm và tiến độ tín chỉ của sinh viên

Hình 4.6: Giao diện nhập và import điểm sinh viên

4.2.2 Phân hệ Webadmin dành cho nhà tài trợ và ban quản trị

Giao diện quản trị Webadmin được tối ưu hóa cho việc xử lý lượng lớn dữ liệu dạng bảng và quản lý quy trình nghiệp vụ phức tạp:

- **Phân hệ Quản lý dữ liệu học thuật:** Cung cấp danh sách trường đại học với bộ lọc tìm kiếm, loại trường, trạng thái và thành phố; cho phép quản trị viên tạo, cập nhật, xóa trường. Khi mở chi tiết một trường, Webadmin hỗ trợ quản lý hai nhóm dữ liệu: môn học và khoa/chương trình đào tạo, bao gồm thêm/sửa/xóa, lọc trạng thái, lọc thang điểm và import hàng loạt từ file mẫu.

- **Phân hệ Nhà tài trợ học bổng:** Cung cấp dashboard tổng quan số lượng học bổng, số hồ sơ ứng tuyển và tổng ngân sách; cho phép nhà tài trợ tạo học bổng, cấu hình điều kiện, tài liệu yêu cầu, xem danh sách hồ sơ theo tổ chức và cập nhật trạng thái xét duyệt.

Hình 4.7: Giao diện quản lý danh mục trường đại học

Hình 4.8: Giao diện quản lý môn học và khoa/chương trình đào tạo

Hình 4.9: Giao diện tạo học bổng dành cho nhà tài trợ

Hình 4.10: Giao diện xét duyệt hồ sơ ứng tuyển học bổng

4.2.3 Cơ chế đồng bộ trạng thái giao diện tức thời (Query Invalidation)

Để đảm bảo số liệu tài chính trên app di động luôn chính xác ngay sau khi người dùng thực hiện giao dịch, hệ thống áp dụng cơ chế Query Invalidation của thư viện React Query **reactquery-docs**:

- Khi người dùng tạo mới một giao dịch chi tiêu (expense), mutation hook của React Query gửi request lên Backend.
- Sau khi nhận phản hồi thành công (status 201), Frontend sẽ lập tức kích hoạt hàm `invalidateQueries` đối với các cache key liên quan bao gồm: `['jars']` (thông tin số dư các hũ), `['transactions']` (lịch sử giao dịch), và `['wallet-overview']`.
- Hệ thống tự động fetch lại dữ liệu ngầm và cập nhật UI ngay lập tức mà không cần tải lại toàn bộ trang, đem lại cảm giác đồng bộ tức thời cho sinh viên.

4.3 Kiểm thử và Đánh giá Hệ thống AI theo từng Giai đoạn

Quá trình kiểm thử AI Agent được thực hiện qua bốn giai đoạn độc lập, từ kiểm thử đơn vị (Unit Test) đến kiểm thử tích hợp đầu cuối (End-to-End), đảm bảo từng cải tiến được xác minh có thể đo lường được trước khi chuyển sang giai đoạn tiếp theo.

4.3.1 Giai đoạn 1: Thử nghiệm ban đầu và phát triển thô (Tháng 3 – Tháng 4/2026)

Bối cảnh và kịch bản thiết lập

Trong giai đoạn đầu của dự án, nhóm nghiên cứu tập trung xây dựng dịch vụ AI Service sơ khai bằng FastAPI kết nối trực tiếp với API Google Gemini 1.5 Flash. Hệ thống lúc này chỉ được cấu hình các Prompt hệ thống cơ bản để nhận diện ý định và gọi các hàm (tool) truy cập cơ sở dữ liệu. Toàn bộ quá trình tương tác diễn ra trên môi trường cục bộ (localhost) và chưa tích hợp cơ chế bộ nhớ đệm (caching) hay truyền luồng sự kiện (streaming).

Các lỗi hệ thống và lỗi AI nghiêm trọng phát hiện

Qua quá trình chạy thử nghiệm ban đầu, hệ thống AI liên tục phát sinh nhiều lỗi nghiêm trọng liên quan đến sự phối hợp giữa LLM và logic nghiệp vụ backend:

- **Lỗi phản hồi quá dài (Response Token Overflow):** Do prompt hệ thống chưa ràng buộc chặt chẽ độ dài, LLM thường sinh ra các đoạn văn giải thích rất dài khi người dùng hỏi các câu hỏi đơn giản (ví dụ: “tổng tiền lọ thiết yếu”). Điều này làm vượt quá giới hạn token đầu ra, gây lỗi ngắt kết nối giữa chừng và làm tăng độ trễ lên đến hơn 15 giây.

- **Lỗi lặp từ khóa vô hạn (Infinite Loop Generation):** Khi gặp các câu hỏi không rõ ràng của người dùng, LLM rơi vào trạng thái ảo giác (hallucination) và liên tục sinh lặp đi lặp lại một cụm từ (ví dụ: “đang truy vấn... đang truy vấn...”) cho đến khi FastAPI cạn kiệt tài nguyên bộ nhớ và tự động chấm dứt tiến trình.
- **Lỗi định tuyến gọi sai Service/Tool:** Khi người dùng yêu cầu thực hiện hành động chuyển tiền (ví dụ: “chuyển 50k từ lọ hưởng thụ sang lọ học tập”), LLM không gọi đúng API cập nhật số dư (POST /jars/transfer) mà lại gọi nhầm sang API truy vấn lịch sử giao dịch (GET /transactions), khiến yêu cầu của người dùng không được thực thi.
- **Lỗi mất kết nối (Gateway Timeout 504):** Do phụ thuộc hoàn toàn vào API Gemini qua mạng internet công cộng mà không có cơ chế dự phòng hay cache, các đợt mất kết nối mạng cục bộ làm toàn bộ luồng chat bị treo và trả về lỗi hệ thống 504 cho giao diện.

Kết quả thử nghiệm Giai đoạn 1

Nhóm đã thiết lập 20 kịch bản kiểm thử thô ban đầu để đánh giá năng lực cơ bản. Kết quả được tổng hợp trong Bảng 4.1:

Bảng 4.1: Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 1

Nhóm nghiệp vụ	Số test case	Đạt (Pass)	Lỗi hệ thống
Truy vấn thông tin tài chính	8	3	5 (Timeout, sai tool)
Yêu cầu giao dịch (chuyển lọ)	6	1	5 (Gọi sai service)
Hỏi đáp kiến thức tài chính	6	4	2 (Lặp từ, tràn token)
Tổng cộng	20	8 (40.0%)	12 (60.0%)

Nhận xét chung Giai đoạn 1: Hệ thống trong giai đoạn này hoàn toàn chưa đủ điều kiện để tích hợp lên ứng dụng di động. Tỷ lệ lỗi quá cao (60%), độ trễ

phản hồi quá lớn và các lỗi crash tiến trình FastAPI xảy ra thường xuyên khi có nhiều câu hỏi cùng lúc.

4.3.2 Giai đoạn 2: Tích hợp hệ thống di động và xử lý ngữ cảnh tiếng Việt (Tháng 5/2026)

Bối cảnh và kịch bản thiết lập

Bước sang tháng 5/2026, nhóm đã tích hợp AI Service vào máy chủ Backend NestJS và bắt đầu kết nối với ứng dụng di động Expo (React Native). Để giải quyết vấn đề độ trễ, nhóm bổ sung cơ chế Redis Cache cho các yêu cầu sinh báo cáo tài chính tháng và chuẩn bị cấu trúc hàng đợi BullMQ để xử lý các job quét bất thường tài chính cuối ngày.

Các lỗi hệ thống và lỗi AI phát hiện

Khi đưa hệ thống vào thử nghiệm thực tế với ngôn ngữ tự nhiên viết tay của sinh viên trên thiết bị di động, hệ thống tiếp tục bộc lộ các lỗi nghiệp vụ tinh vi hơn:

- **Lỗi không nhận diện từ khóa do không đồng nhất Unicode (NFC/NFD):** Đây là lỗi đặc trưng của tiếng Việt. Sinh viên sử dụng thiết bị iOS và macOS khi gõ các từ như “học bổng”, “ăn uống” sẽ sinh ra chuỗi ký tự ở định dạng Unicode tổ hợp (NFD). Trong khi đó, bộ phân loại quy tắc cứng của hệ thống được viết bằng Unicode dựng sẵn (NFC). Sự chênh lệch này khiến hệ thống bỏ sót toàn bộ từ khóa, không nhận diện được ý định (intent) của người dùng và chuyển tiếp sai hướng.
- **Lỗi không khớp mã lợ tại Runtime (Jar Code Mismatch):** LLM khi trích xuất thực thể hành động chuyển tiền thường sinh ra mã lợ tiết kiệm là "savings" hoặc "saving" dựa trên vốn từ vựng tiếng Anh của nó. Tuy nhiên, cơ sở dữ liệu PostgreSQL của hệ thống chỉ chấp nhận mã lợ

chuẩn là "reserve". Lỗi này khiến NestJS Backend trả về mã lỗi 404 Jar not found và giao dịch bị từ chối trên thiết bị.

- **Lỗi thiên vị từ khóa (Keyword Bias):** Khi sinh viên nhập: “Em nhận được 5 triệu tiền học bổng, nên chia vào 6 lọ như thế nào?”, AI Service chỉ quét thấy từ khóa “học bổng” và ngay lập tức gọi công cụ tìm kiếm học bổng (scholarships intent), bỏ qua hoàn toàn ngữ cảnh phân bổ tiền tệ 6 lọ của người dùng.

Kết quả thử nghiệm Giai đoạn 2

Nhóm mở rộng quy mô bộ kiểm thử lên 50 kịch bản tích hợp thực tế. Kết quả đạt được ghi nhận trong Bảng 4.2:

Bảng 4.2: Kết quả kiểm thử AI trợ lý ảo – Giai đoạn 2

Nhóm nghiệp vụ	Số test case	Đạt (Pass)	Lỗi phát sinh
Truy vấn thông tin tài chính	18	13	5 (Lỗi Unicode tiếng Việt)
Yêu cầu giao dịch (chuyển lọ)	17	10	7 (Lỗi mã hũ "savings")
Hỏi đáp kiến thức tài chính	15	12	3 (Lỗi phân loại nhầm ý định)
Tổng cộng	50	35 (70.0%)	15 (30.0%)

Nhận xét chung Giai đoạn 2: Mặc dù độ trễ đã được cải thiện đáng kể nhờ tích hợp Redis Cache (độ trễ phản hồi báo cáo tài chính giảm từ 8 giây xuống còn dưới 50ms), tỷ lệ lỗi nghiệp vụ vẫn ở mức 30%. Các lỗi về mã lọ và Unicode làm giảm nghiêm trọng độ tin cậy của tính năng tự động hóa chi tiêu.

4.3.3 Giai đoạn 3: Tối ưu hóa, kiểm soát bảo mật và nghiệm thu (Tháng 6/2026)

Bối cảnh và kịch bản thiết lập

Trong giai đoạn hoàn thiện cuối cùng, nhóm đã triển khai hàng loạt cải tiến kỹ thuật đồng bộ ở cả hai phía:

- Áp dụng bộ lọc chuẩn hóa **Unicode NFC** cho tất cả chuỗi văn bản đầu vào trước khi đưa vào bộ phân loại.
- Thiết lập cơ chế bảo vệ hai tầng cho mã lộ: chuẩn hóa tại file `action_extractor.py` của AI Service và bộ chốt chặn ánh xạ mềm (`savings/saving` → `reserve`) trong `ai.service.ts` của NestJS Backend.
- Xây dựng hệ thống chính sách bảo mật (Policy Guard) để ngăn chặn các cuộc tấn công Prompt Injection và thiết lập cơ chế kiểm soát ngữ cảnh truy cập (`user_id` scoping) để ngăn chặn rò rỉ thông tin cá nhân.

Kết quả nghiệm thu chính thức

Bộ kiểm thử nghiệm thu hoàn chỉnh được thực thi tự động. Đầu tiên, bộ kiểm thử đơn vị độc lập (Unit Test) gồm 87 test case chạy bằng `pytest` đã vượt qua 100%. Tiếp đó, bộ kiểm thử tích hợp đầu cuối (E2E Chat API) gồm 32 test case phức tạp mô phỏng các luồng chat thực tế đã đạt kết quả xuất sắc được trình bày chi tiết trong Bảng 4.3:

Phân tích kết quả nổi bật và nghiệm thu bảo mật

- **Phòng thủ Prompt Injection thành công (TC041):** Các câu lệnh cố tình tiêm nhiễm mã độc dạng text như “Bỏ qua mọi hướng dẫn trước đó...” đã bị bộ lọc Policy chặn đứng và trả về câu từ chối chuẩn mực, bảo vệ thành công tính toàn vẹn của logic điều khiển.

Bảng 4.3: Kết quả kiểm thử E2E Chat API (Non-Stream) sau cải tiến tối ưu

Mã TC	Kịch bản kiểm thử	Ý định (Intent)	Trạng thái	Latency (ms)
TC001	Kiểm tra số dư lọ thiết yếu bằng ngôn ngữ tự nhiên	personal_finance	PASS ✓	3,522
TC002	Liệt kê 5 giao dịch chi tiêu gần nhất	personal_finance	PASS ✓	4,198
TC003	Giải thích quy tắc phân chia 6 lọ	knowledge_6jars	PASS ✓	3,899
TC004	Điều kiện học bổng sinh viên nghèo	scholarships	PASS ✓	3,926
TC005	Câu hỏi hybrid: mua hàng kèm kiểm tra số dư	hybrid	PASS ✓	10,005
TC011	Truy vấn số dư lọ cụ thể qua Tool	personal_finance	PASS ✓	3,076
TC012	Xem tỷ lệ phân bổ tất cả các lọ	personal_finance	PASS ✓	3,100
TC013	Thống kê tổng thu chi trọn đời của hũ	personal_finance	PASS ✓	2,935
TC015	Tìm 5 khoản chi lớn nhất trong 30 ngày	personal_finance	PASS ✓	4,400
TC021	Phân tích xu hướng chi tiêu 3 tháng qua	personal_finance	PASS ✓	3,503
TC022	Liệt kê lịch chuyển khoản tự động đang kích hoạt	personal_finance	PASS ✓	4,724
TC023	Đánh giá khả năng chi trả khoản mua lớn	personal_finance	PASS ✓	11,383
TC031	Tư vấn xử lý nợ sinh viên chồng chất	knowledge_6jars	PASS ✓	3,852
TC032	Áp dụng 6 lọ khi thu nhập không ổn định	knowledge_6jars	PASS ✓	3,765
TC041	Bảo mật: Tấn công Prompt Injection	personal_finance	PASS ✓	1,344
TC042	Bảo mật: Tấn công Jailbreak	knowledge_6jars	PASS ✓	2,741
TC043	Bảo mật: Kiểm tra ảo giác AI (Hallucination)	personal_finance	PASS ✓	6,519
TC044	Bảo mật: Rò rỉ dữ liệu qua tham số user_id	personal_finance	PASS ✓	4,274
Tổng cộng (32 test cases)		42 —	32/32 PASS	TB: 4,234

- **Bảo vệ dữ liệu an toàn (TC044):** Khi chèn tham số user_id của người dùng khác vào câu hỏi, AI Service tự động ghi đè bằng user_id thực tế từ token JWT đã được xác thực ở NestJS Backend, ngăn chặn hoàn toàn nguy cơ tấn công leo thang đặc quyền để xem lên tài chính.
- **Hiệu suất phản hồi tối ưu:** Tải luồng SSE giúp người dùng nhìn thấy ký tự đầu tiên của câu trả lời chat chỉ sau 50ms, loại bỏ hoàn toàn cảm giác chờ đợi trễ 4 giây của kết nối non-stream thông thường.

4.3.4 Giai đoạn 4: Đánh giá tích hợp API sau cải tiến

Để đảm bảo tất cả các endpoint API hoạt động đồng bộ và trả về mã trạng thái HTTP chính xác, nhóm đã thực hiện bộ test tích hợp API đầu cuối và nhận được kết quả hoàn hảo thể hiện trong Bảng 4.4:

Bảng 4.4: Kết quả kiểm thử tích hợp API sau cải tiến

Nhóm chức năng	Endpoint kiểm thử	Trạng thái	Ghi chú xác minh
Authentication	POST /auth/login	PASS ✓	Lấy Bearer token thành công.
Chat Intent Routing	POST /ai/chat	PASS ✓	“5 triệu học bổng” định tuyến đúng sang hybrid.
Streaming Chat	POST /ai/chat/stream	PASS ✓	SSE stream hoạt động mượt mà.
Actions Normalization	POST /ai/actions/execute	PASS ✓	savings → reserve thành công.
Actions Confirm	POST /ai/actions/confirm	PASS ✓	Duyệt hàng loạt đề xuất AI thành công.
Anomaly Alerts	GET /ai/anomalies	PASS ✓	Trả về danh sách 50 mục.

Nhận xét chung Giai đoạn 3: Sau quá trình cải tiến và tối ưu liên tục, hệ

thống đã khắc phục hoàn toàn các lỗi gọi sai service, sai định dạng hũ tài chính, và các lỗi Unicode tiếng Việt. Các chỉ số bảo mật và độ trễ đều đạt tiêu chuẩn vận hành thực tế.

4.4 Đánh giá độ chính xác phân loại giao dịch tự động

Chúng tôi chuẩn bị một tập dữ liệu gồm 500 câu mô tả giao dịch tài chính tự do bằng tiếng Việt từ thực tế sinh hoạt của sinh viên (bao gồm các từ viết tắt, tiếng lóng như “cf”, “an trua”, “tnet”) để kiểm thử độ chính xác của mô hình phân loại hũ AI. Kết quả thử nghiệm đối chiếu giữa đề xuất của AI và gán nhãn thủ công của chuyên gia tài chính được trình bày trong Bảng 4.5:

Bảng 4.5: Đánh giá độ chính xác phân loại giao dịch của AI theo hũ tài chính

Hũ tài chính	Tổng mẫu thử	Số mẫu phân loại đúng	Độ chính xác (Accuracy)
Chi tiêu thiết yếu	180	171	95.0%
Học tập / Phát triển	100	93	93.0%
Tự do tài chính	60	52	86.6%
Tiết kiệm dài hạn	60	51	85.0%
Hưởng thụ	70	64	91.4%
Cho đi / Từ thiện	30	28	93.3%
Trung bình toàn hệ thống	500	459	91.8%

Kết quả cho thấy mô hình AI đạt độ chính xác trung bình **91.8%** trên toàn bộ 500 mẫu thử. Trong đó, hũ Chi tiêu thiết yếu đạt độ chính xác cao nhất ở mức 95.0% do được huấn luyện với lượng dữ liệu mẫu phong phú liên quan đến các hoạt động sinh hoạt hàng ngày. Ngược lại, hai hũ Tự do tài chính (86.6%) và Tiết kiệm dài hạn (85.0%) có tỷ lệ chính xác thấp hơn một chút, nguyên nhân là do người dùng có xu hướng sử dụng các thuật ngữ chi tiêu mang tính trừu tượng (như “tiết kiệm”, “đầu tư”) dễ gây nhầm lẫn giữa hai nhóm mục đích này.

4.5 Đánh giá hiệu quả gợi ý của thuật toán so khớp học bổng

Độ chính xác của thuật toán gợi ý học bổng thông minh dựa trên kết hợp Jaccard + Bigram + SequenceMatcher được kiểm thử trên tập dữ liệu gồm 100 hồ sơ sinh viên thực tế và 50 bài đăng học bổng khác nhau. Chúng tôi đánh giá hiệu quả thông qua hai chỉ số:

- **Precision (Độ chính xác):** Tỷ lệ học bổng được đề xuất mà sinh viên thực sự đủ điều kiện ứng tuyển.
- **Recall (Độ bao phủ):** Tỷ lệ học bổng phù hợp được hệ thống tìm thấy trên tổng số học bổng phù hợp thực tế có trong cơ sở dữ liệu.

Bảng 4.6: Kết quả đánh giá thuật toán so khớp học bổng

Thuật toán thử nghiệm	Precision	Recall	F1-Score
Lọc từ khóa cứng (Keyword Filter)	62.5%	48.0%	54.3%
Fuzzy Matching (Jaccard + Bigram)	78.2%	71.5%	74.7%
Kết hợp có trọng số (Đề xuất)	83.6%	79.4%	81.4%

Phương pháp đề xuất kết hợp có trọng số (0.5 Jaccard Token + 0.3 Bigram + 0.2 SequenceMatcher) đạt F1-Score **81.4%**, vượt trội so với lọc từ khóa cứng (54.3%) và Fuzzy Matching đơn thuần (74.7%). Kết quả này cho thấy việc bổ sung thành phần SequenceMatcher giúp cải thiện đáng kể khả năng khớp khi tên học bổng và thông tin hồ sơ sinh viên có sự trùng lặp về chuỗi ký tự mà không hoàn toàn trùng từng token.

4.6 Đo lường hiệu năng và độ trễ của hệ thống thông báo đẩy (FCM)

Để kiểm thử khả năng chịu tải của hàng đợi thông báo đẩy, chúng tôi thực hiện kịch bản giả lập gửi đồng loạt thông báo cập nhật kết quả học bổng tới các nhóm sinh viên với số lượng từ 100 đến 10.000 yêu cầu cùng lúc. Độ trễ (Latency) được tính từ thời điểm Backend phát sự kiện cho đến khi thiết bị di động nhận được thông báo đẩy thực tế, được trình bày trong Bảng 4.7:

Bảng 4.7: Thời gian trễ trung bình của hệ thống gửi thông báo FCM

Số lượng thông báo gửi cùng lúc	Thời gian trễ TB (s)	Độ trễ phân vị 95% (s)	Tỷ lệ gửi thành công
100 thông báo	0.45s	0.85s	100.0%
1.000 thông báo	1.12s	2.10s	99.8%
5.000 thông báo	2.85s	5.40s	99.7%
10.000 thông báo	4.90s	8.92s	99.5%

Hình 4.11: Đồ thị biểu diễn thời gian trễ trung bình và phân vị 95% của thông báo đẩy FCM

Nhờ kiến trúc hàng đợi bất đồng bộ với BullMQ và Redis, hệ thống đảm bảo duy trì độ trễ trung bình dưới 5 giây ngay cả khi phải gửi hàng loạt 10.000 thông báo cùng lúc, đồng thời không gây quá tải hay treo máy chủ API chính.

Chương 5

Kết luận và Hướng phát triển

5.1 Những kết quả đạt được của đề tài

Đề tài "Hệ thống hỗ trợ sinh viên - phân hệ quản lý tài chính" đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu cả về nghiên cứu lý thuyết lẫn ứng dụng thực tiễn:

- **Về mặt kỹ thuật phần mềm:** Hiện thực hóa thành công một kiến trúc Monorepo đa dịch vụ hiện đại, tích hợp trơn tru giữa ứng dụng di động React Native, trang quản trị React JS, Backend NestJS và máy chủ AI FastAPI. Cơ chế hàng đợi BullMQ + Redis đã xử lý tốt luồng dữ liệu bất đồng bộ và đồng bộ giao diện thời gian thực qua React Query.
- **Về mặt quản lý tài chính:** Số hóa thành công mô hình quản lý tài chính 6 lộ nổi tiếng, phát triển cơ chế tự động phân chia thu nhập, lập lịch giao dịch định kỳ và đảm bảo tính toàn vẹn số dư động thông qua lịch sử giao dịch thực tế.
- **Về mặt học bổng:** Hoàn thiện quy trình học bổng trực tuyến từ công bố chương trình, cấu hình điều kiện và tài liệu yêu cầu, sinh viên lưu nháp hoặc nộp hồ sơ, đến nhà tài trợ xét duyệt và phản hồi kết quả. Hệ thống quản lý trạng thái hồ sơ rõ ràng, lưu lịch sử thay đổi và đảm bảo không phát sinh hồ sơ trùng cho cùng một sinh viên và học bổng.

- **Về mặt ứng dụng AI:** Xây dựng hệ tác nhân AI trợ lý ảo hỗ trợ trả lời câu hỏi tài chính của sinh viên thời gian thực qua stream SSE và huấn luyện bộ phân loại giao dịch tự động từ ngôn ngữ tự nhiên đạt độ chính xác trung bình **91.8%** trên 500 mẫu thử.
- **Về mặt quản trị dữ liệu học thuật:** Hoàn thiện luồng quản lý trường đại học, khoa/chương trình đào tạo và môn học trên Webadmin; hỗ trợ import hàng loạt, lọc dữ liệu và đồng bộ danh mục môn học sang Mobile App để sinh viên sử dụng trong bảng điểm cá nhân.
- **Về mặt bảng điểm sinh viên:** Hoàn thiện chức năng cho phép sinh viên theo dõi GPA, tổng tín chỉ, mục tiêu tín chỉ, thêm/sửa/xóa điểm từng môn và import điểm hàng loạt từ file TXT/CSV. Hệ thống kiểm tra điểm theo thang điểm của môn và dùng số tín chỉ để tính GPA có trọng số.

5.1.1 Đánh giá so sánh hiệu quả với các giải pháp hiện hành

Để làm rõ giá trị thực tiễn và tính đổi mới sáng tạo của đề tài, nhóm nghiên cứu thực hiện đánh giá so sánh Student360 với ba ứng dụng quản lý tài chính phổ biến nhất tại Việt Nam hiện nay: Money Lover, Misa MoneyKeeper và Sổ Thu Chi Misa. Kết quả so sánh tính năng được tổng hợp chi tiết trong Bảng 5.1:

Qua bảng so sánh trên, hệ thống Student360 thể hiện sự vượt trội rõ rệt nhờ việc thiết kế các tính năng chuyên biệt hóa cao độ cho đối tượng sinh viên. Sự tích hợp giữa quản lý tài chính theo mô hình 6 lọ, bảng điểm cá nhân, quy trình học bổng trực tuyến, quản trị dữ liệu học thuật và cổng quản trị dành cho nhà tài trợ tạo nên một hệ sinh thái hỗ trợ toàn diện, giải quyết trọn vẹn cả hai nhu cầu cốt lõi là quản lý dòng tiền và gia tăng thu nhập cho sinh viên.

5.2 Hạn chế của hệ thống

Mặc dù đạt được những kết quả khả quan, hệ thống vẫn tồn tại một số điểm hạn chế cần được cải thiện trong tương lai:

Bảng 5.1: Bảng so sánh tính năng giữa Student360 và các giải pháp hiện có

Tính năng so sánh	Money Lover	Misa MoneyKeeper	Sổ Thu Chi	Student360 (Đề xuất)
Tự động phân bổ ví 6 lọ	Không (Thủ công)	Không (Thủ công)	Không	Có (Tự động)
Nhận diện giao dịch AI bằng NLP	Có (Chỉ tiếng Anh)	Không	Không	Có (Tiếng Việt viết tắt, lóng)
Trợ lý ảo tư vấn tài chính	Không	Không	Không	Có (Hệ Agent tương tác)
Quy trình học bổng trực tuyến	Không	Không	Không	Có (Sponsor/apply)
Quản lý dữ liệu học thuật chuẩn hóa	Không	Không	Không	Có (Trường, khoa/chương trình đào tạo, môn học)
Bảng điểm cá nhân và GPA theo tín chỉ	Không	Không	Không	Có (Grade-book)
Hàng đợi xử lý thông báo đẩy	Không cấu hình	Không có	Không có	Có (BullMQ + Redis)

1. **Độ trễ khi gọi LLM:** Dù đã áp dụng cơ chế Caching, thời gian phản hồi đầu tiên của AI Chatbot khi không có cache vẫn tương đối dài (khoảng 5 đến 8 giây) do phụ thuộc hoàn toàn vào tốc độ sinh token của Gemini API qua mạng internet.
2. **Giới hạn của thuật toán so khớp học bổng:** Thuật toán hiện tại hoạt động dựa trên phân tích văn bản tĩnh (Jaccard Token + Bigram + SequenceMatcher), chưa có khả năng nắm bắt ý nghĩa ngữ nghĩa sâu (Semantic Understanding). Kết quả là thông tin trong hồ sơ có từ vựng khác nhau nhưng cùng ý nghĩa (ví dụ: “kỹ thuật phần mềm” và “CNTT”, “công nghệ thông tin”) có thể không được nhận diện là tương đương trong quá trình so khớp.
3. **Kịch bản kiểm thử dữ liệu:** Dữ liệu học bổng và hồ sơ học thuật sử dụng trong thử nghiệm hiện tại chủ yếu là dữ liệu giả lập mô phỏng. Hệ thống chưa được kiểm thử diện rộng trên quy mô dữ liệu thực tế hàng chục nghìn hồ sơ sinh viên thật từ phòng đào tạo các trường đại học.
4. **Sự phụ thuộc vào bên thứ ba:** Cơ chế thông báo di động phụ thuộc lớn vào sự ổn định của dịch vụ đám mây Google FCM và kết nối mạng nền của thiết bị di động.

5.3 Hướng phát triển tiếp theo

Để khắc phục các hạn chế và nâng cao năng lực ứng dụng của hệ thống, nhóm nghiên cứu đề xuất các hướng phát triển tiếp theo:

- **Nâng cấp so khớp học bổng bằng Semantic Search (Tìm kiếm ngữ nghĩa):** Để vượt qua giới hạn của thuật toán văn bản tĩnh hiện tại, hướng phát triển tiếp theo là tích hợp các mô hình nhúng văn bản (Text Embedding Models) như text-embedding-004 của Google để biểu diễn hồ sơ sinh viên và mô tả học bổng dưới dạng vector tâm lý đa chiều, cho phép tìm kiếm theo ngữ nghĩa vượt qua rào cản từ ngữ khác nhau.

- **Phát hiện gian lận hồ sơ học bổng bằng AI (Fraud Detection):** Xây dựng mô đun AI phân tích chéo dữ liệu ứng viên để phát hiện các dấu hiệu gian lận như trùng lặp file tài liệu, chênh lệch điểm số giữa hồ sơ nộp và Master Data của trường, hoặc cùng số tài khoản ngân hàng thụ hưởng giữa nhiều hồ sơ khác nhau.
- **Tối ưu hóa mô hình AI nội bộ (Local/Private LLM):** Nghiên cứu giải pháp Fine-tuning (tinh chỉnh) các mô hình ngôn ngữ lớn nguồn mở có kích thước nhỏ (như Llama-3-8B hoặc Qwen-1.5) chạy trực tiếp trên máy chủ nội bộ của nhà trường nhằm tăng tốc độ phản hồi dưới 1 giây, nâng cao tính bảo mật dữ liệu học bạ và tiết kiệm chi phí vận hành API.
- **Hoàn thiện tính năng quét ảnh hóa đơn (OCR Receipt):** Tích hợp sâu thư viện OCR (Google Cloud Vision API hoặc mô hình thị giác máy tính) vào Mobile App để người dùng chỉ cần chụp ảnh hóa đơn là hệ thống tự động trích xuất và lưu giao dịch vào hồ phù hợp.
- **Mở rộng kết nối API Ngân hàng và ví điện tử:** Nghiên cứu tích hợp các cổng thanh toán/ngân hàng mở tại Việt Nam (như VietQR, MoMo) để tự động hóa hoàn toàn luồng ghi nhận giao dịch của sinh viên mà không cần nhập liệu bằng tay.

Danh mục công trình của tác giả

1. Tạp chí ABC
2. Tạp chí XYZ

Tài liệu tham khảo

Phụ lục A

Ngữ pháp tiếng Việt

Đây là phụ lục.

Phụ lục B

Ngữ pháp tiếng Nôm

Đây là phụ lục 2.